AI and Advanced Machine Learning

Module II: **Introduction to Deep Learning: MLPs and CNNs for Computer Vision**

Yan Kyaw Tun

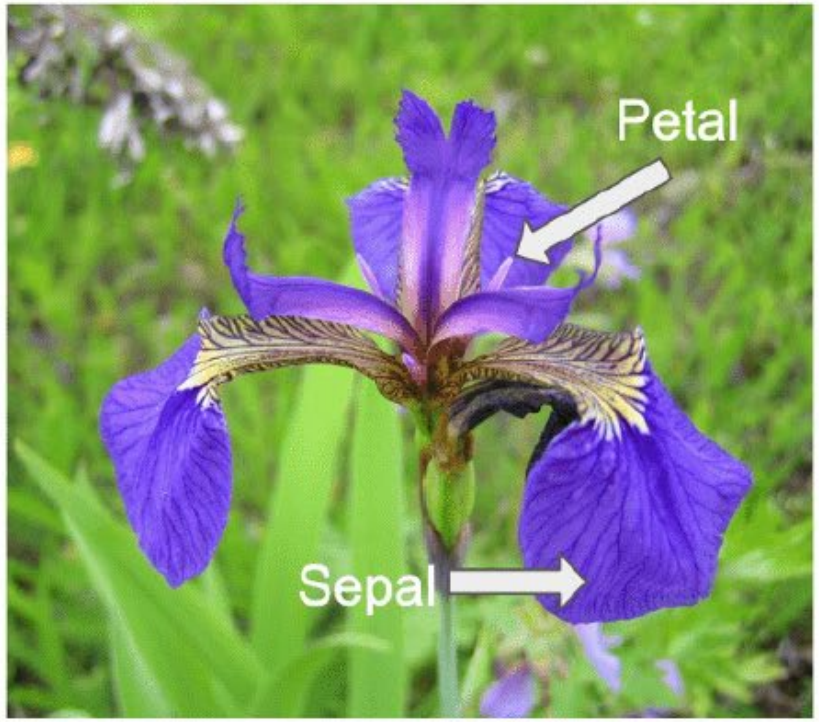
Tenure Track Assistant Professor

Edge Computing and Networking Group

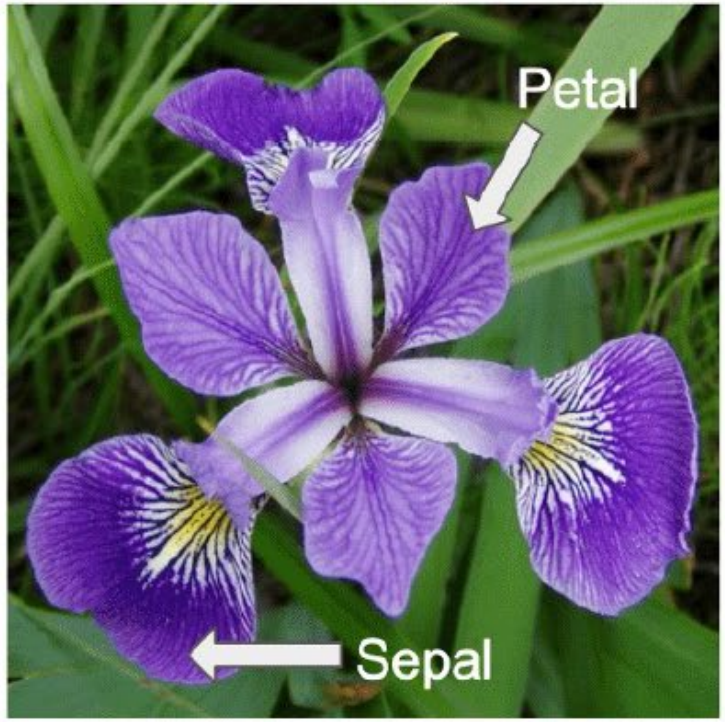
AALBORG UNIVERSITY

Neural Networks

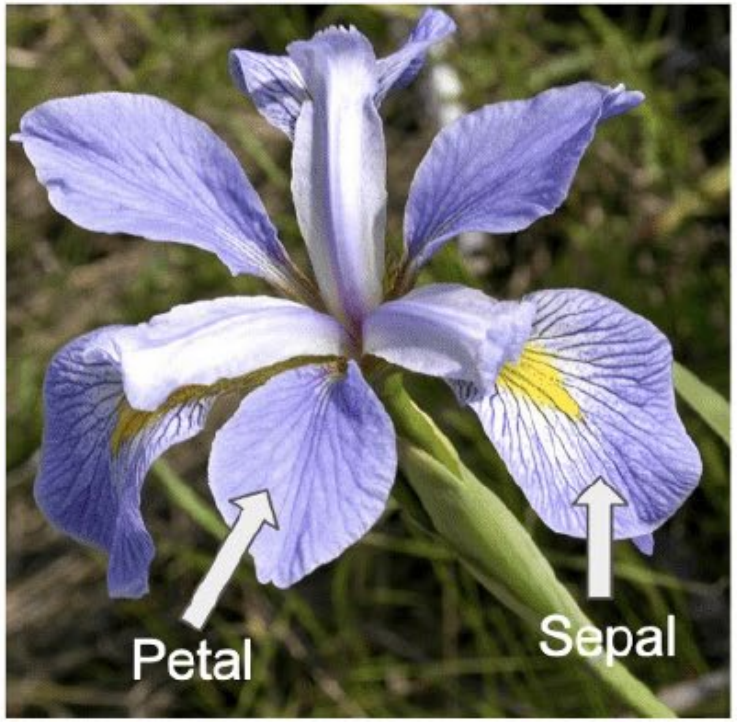
Iris setosa



Iris versicolor



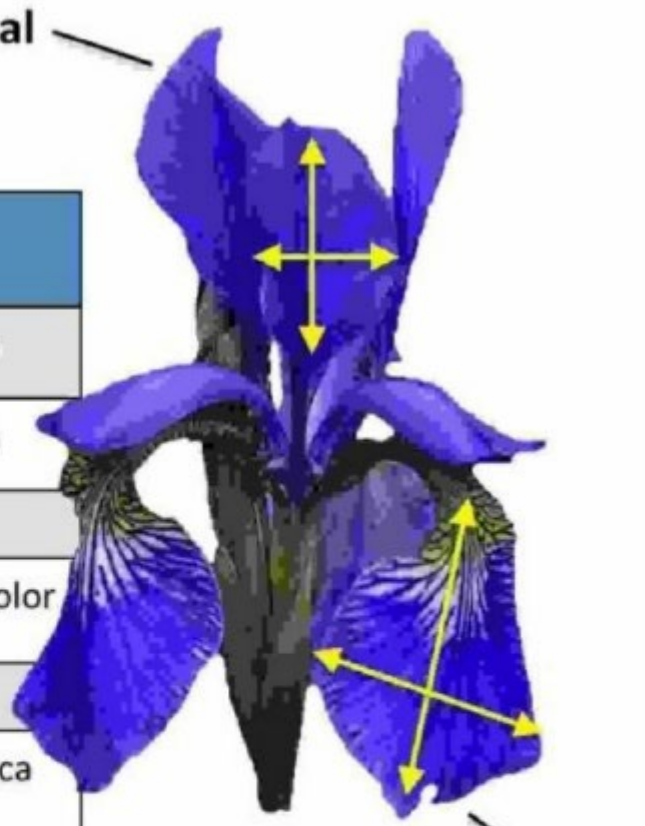
Iris virginica



Basic Terminology and Notations

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
			...		
50	6.4	3.5	4.5	1.2	Versicolor
			...		
150	5.9	3.0	5.0	1.8	Virginica



Features
(attributes, measurements, dimensions)

Class labels
(targets)

Basic Terminology and Notations

The Iris dataset consisting of 150 samples and four features can then be written as a

150×4 matrix $\boldsymbol{X} \in \mathbb{R}^{150 \times 4}$:

$$\begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(150)} & x_2^{(150)} & x_3^{(150)} & x_4^{(150)} \end{bmatrix}$$

We use lowercase, bold-face letters to refer to vectors $\left(\boldsymbol{x} \in \mathbb{R}^{n \times 1}\right)$ and uppercase, bold-face letters to refer to matrices $(\boldsymbol{X} \in \mathbb{R}^{n \times m})$. To refer to single elements in a vector or matrix, we write the letters in italics ($x^{(n)}$ or $x_{(m)}^{(n)}$, respectively).

For example, x_1^{150} refers to the first dimension of flower sample 150, the *sepal length*. Thus, each row in this feature matrix represents one flower instance and can be written as a four-dimensional row vector $\boldsymbol{x}^{(i)} \in \mathbb{R}^{1 \times 4}$:

Basic Terminology and Notations

$$\boldsymbol{x}^{(i)} = \begin{bmatrix} x_1^{(i)} & x_2^{(i)} & x_3^{(i)} & x_4^{(i)} \end{bmatrix}$$

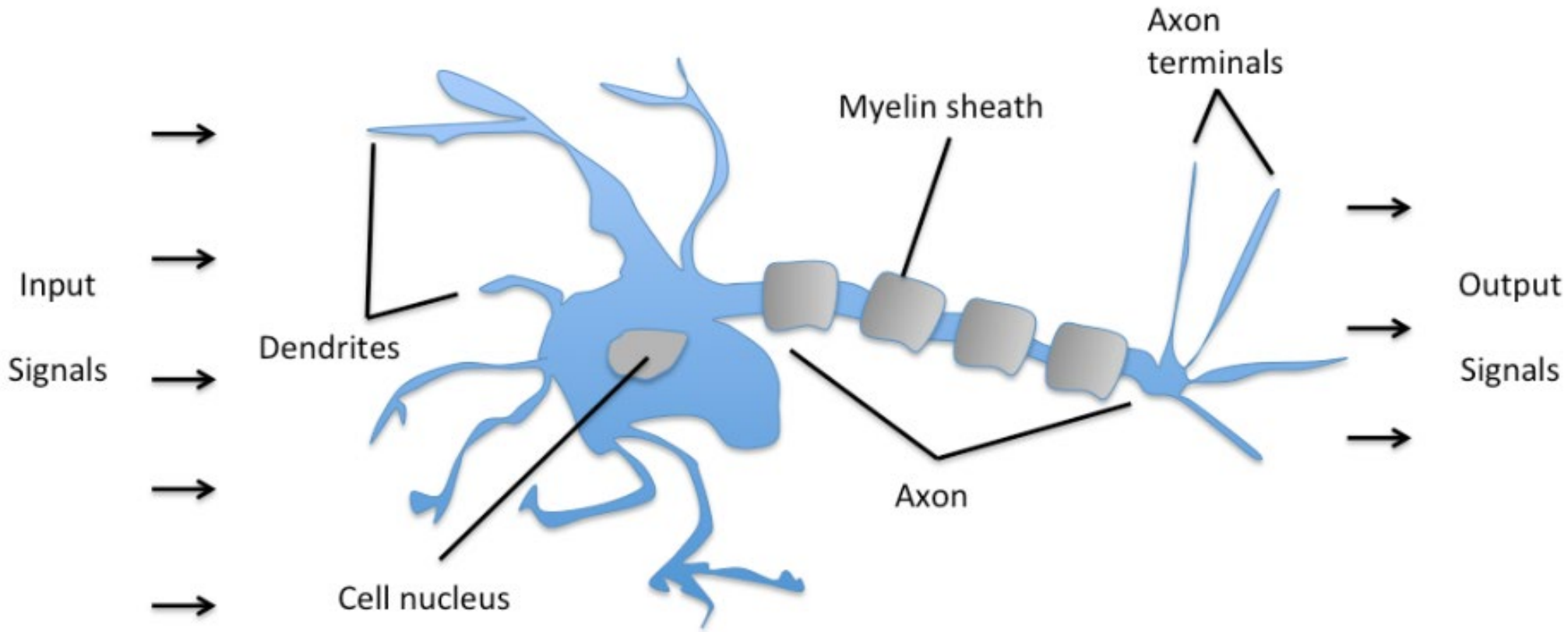
And each feature dimension is a 150-dimensional column vector $\boldsymbol{x}_j \in \mathbb{R}^{150\times 1}$. For example:

$$\boldsymbol{x}_j = \begin{bmatrix} x_j^{(1)} \\ x_j^{(2)} \\ \vdots \\ x_j^{(150)} \end{bmatrix}$$

Similarly, we store the target variables (here, class labels) as a 150-dimensional column vector:

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ \dots \\ y^{(150)} \end{bmatrix} \left(y \in \{\text{Setosa},\ \text{Versicolor},\ \text{Virginica}\} \right)$$

Perceptron for Classification



Schematic of a biological neuron.

Perceptron for Classification

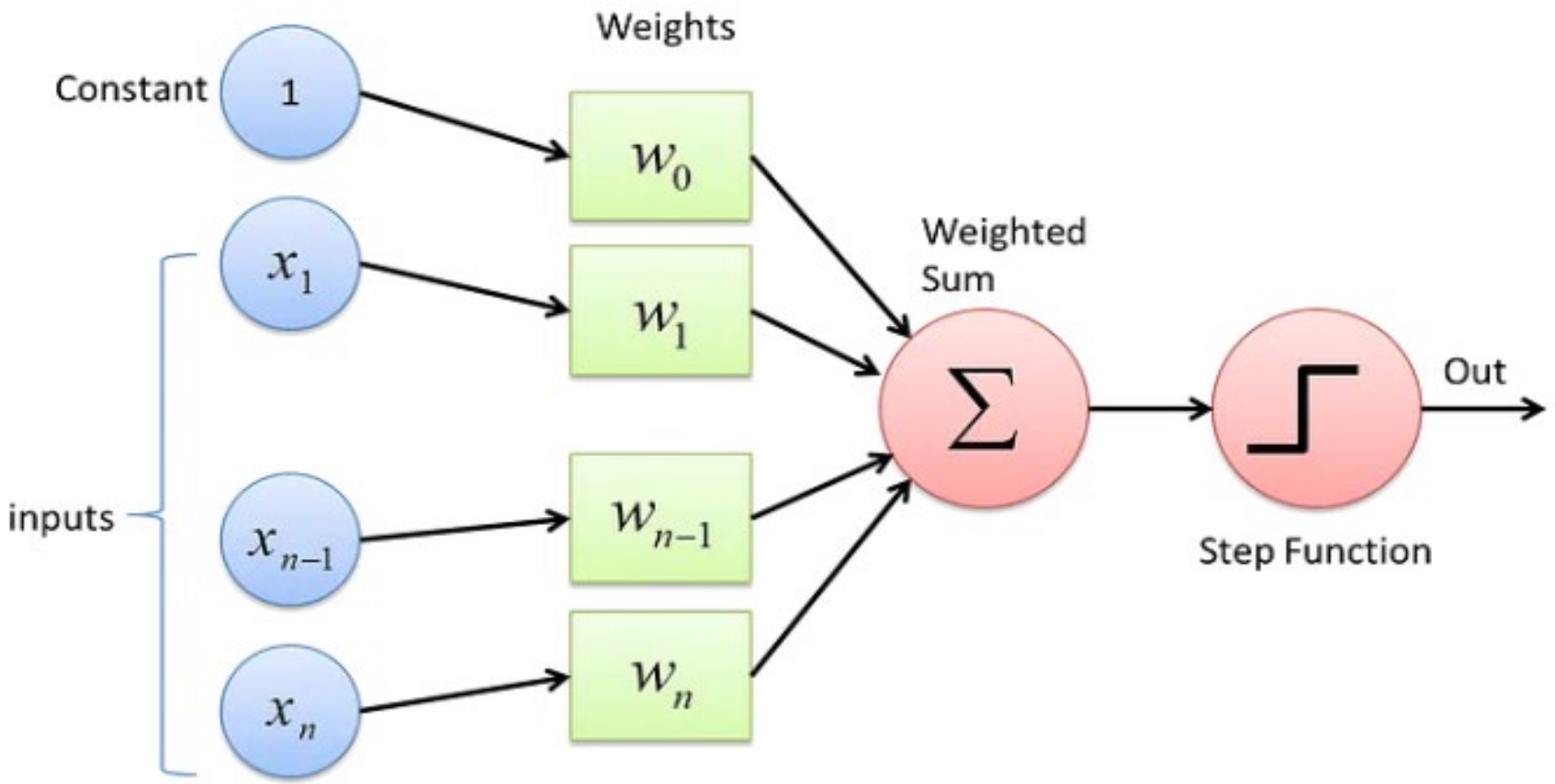


Fig : Perceptron

- Warren McCullock and Walter Pitts published the first concept of a simplified brain cell, the so-called **McCullock-Pitts** (MCP) neuron (**Perceptron**) in 1943,

Perceptron for Classification

More formally, we can put the idea behind **artificial neurons** into the context of a binary classification task where we refer to our two classes as 1 (positive class) and -1 (negative class) for simplicity. We can then define a decision function ($\phi(z)$) that takes a linear combination of certain input values **x** and a corresponding weight vector **w**, where z is the so-called net input $z = w_1 x_1 + \ldots + w_m x_m$:

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

$$\phi(z) = \begin{cases} 1 \ \ if \ z \geq \theta \\ -1 \ \ otherwise \end{cases}$$

Perceptron for Classification

For simplicity, we can bring the threshold θ to the left side of the equation and define a weight-zero as $w_0 = -\theta$ and $x_0 = 1$ so that we write z in a more compact form:

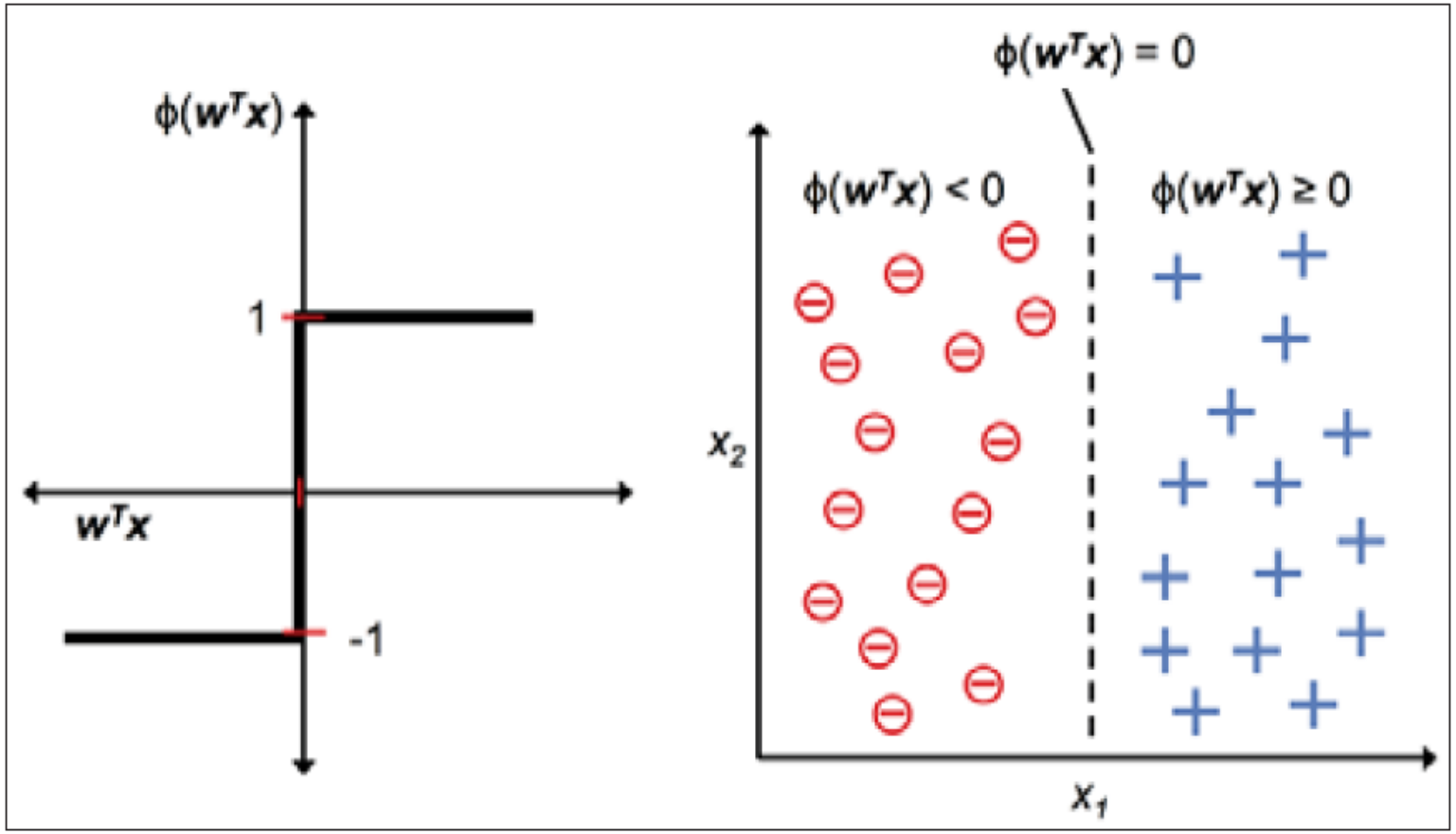
$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

And:

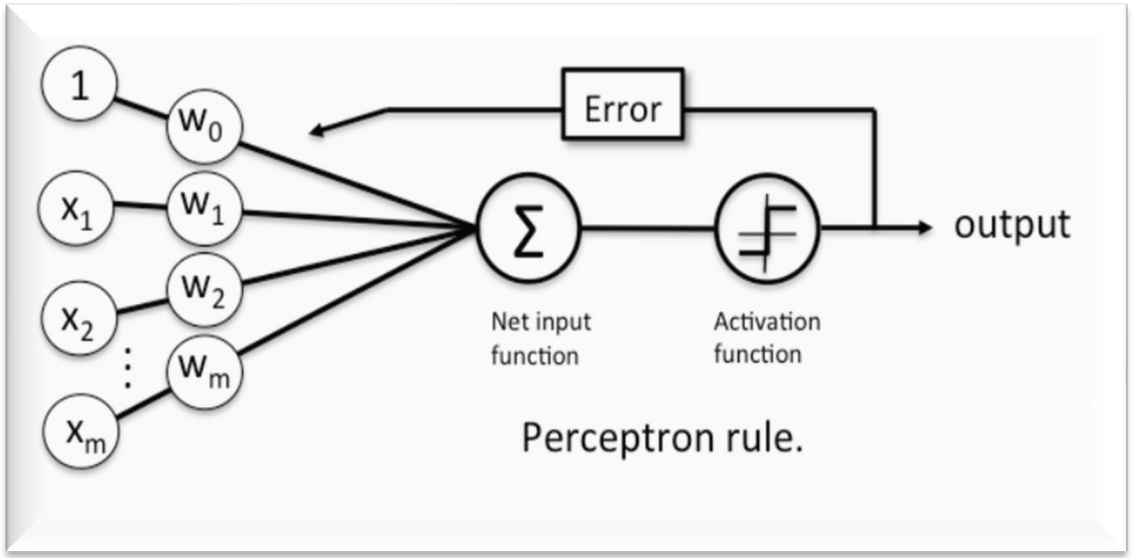
$$\phi(z) = \begin{cases} 1 \ \ if \ z \geq 0 \\ -1 \ \ otherwise \end{cases}$$

In machine learning literature, the negative threshold, or weight, $w_0 = -\theta$, is usually called the **bias unit**.

Perceptron for Classification



Perceptron for Classification



Perceptron Learning Rule by Frank Rosenblatt

1. Initialize the weights to 0 or small random numbers.
2. For each training sample $\boldsymbol{x}^{(i)}$:
   a. Compute the output value $\hat{y}$.
   b. Update the weights.

Weight Update

$$w_j := w_j + \Delta w_j$$

$$\Delta w_j = \eta\left(y^{(i)} - \hat{y}^{(i)}\right)x_j^{(i)}$$

Predicted class label

Learning rate

True class label

Examples of Weight Update

$$\Delta w_j = \eta\left(-1-\left(-1\right)\right)x_j^{(i)} = 0$$

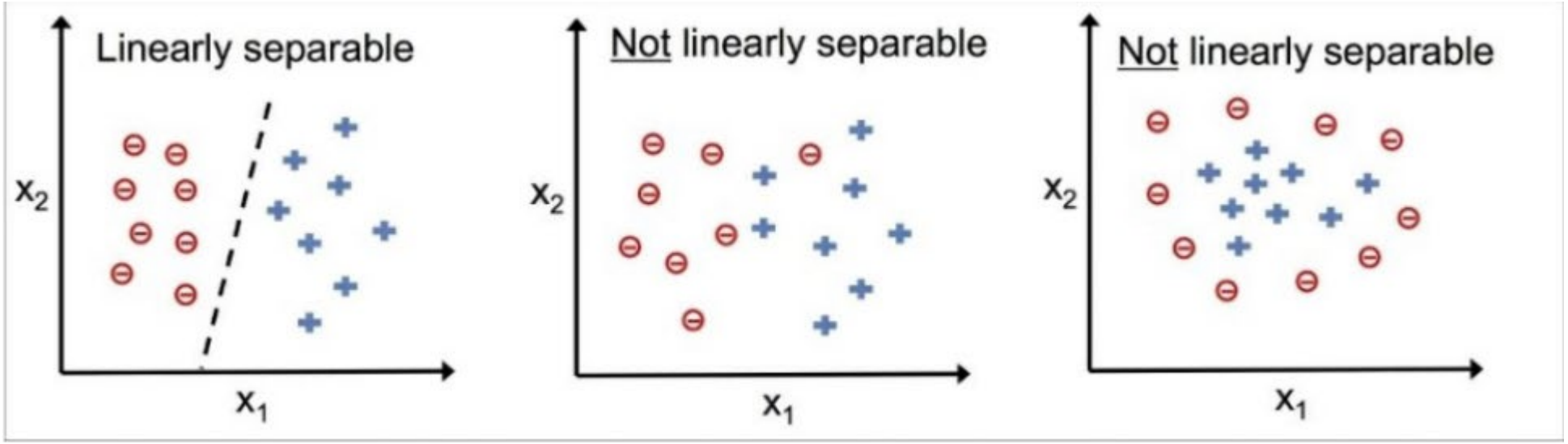
$$\Delta w_j = \eta\left(1-1\right)x_j^{(i)} = 0$$

Perceptron for Classification

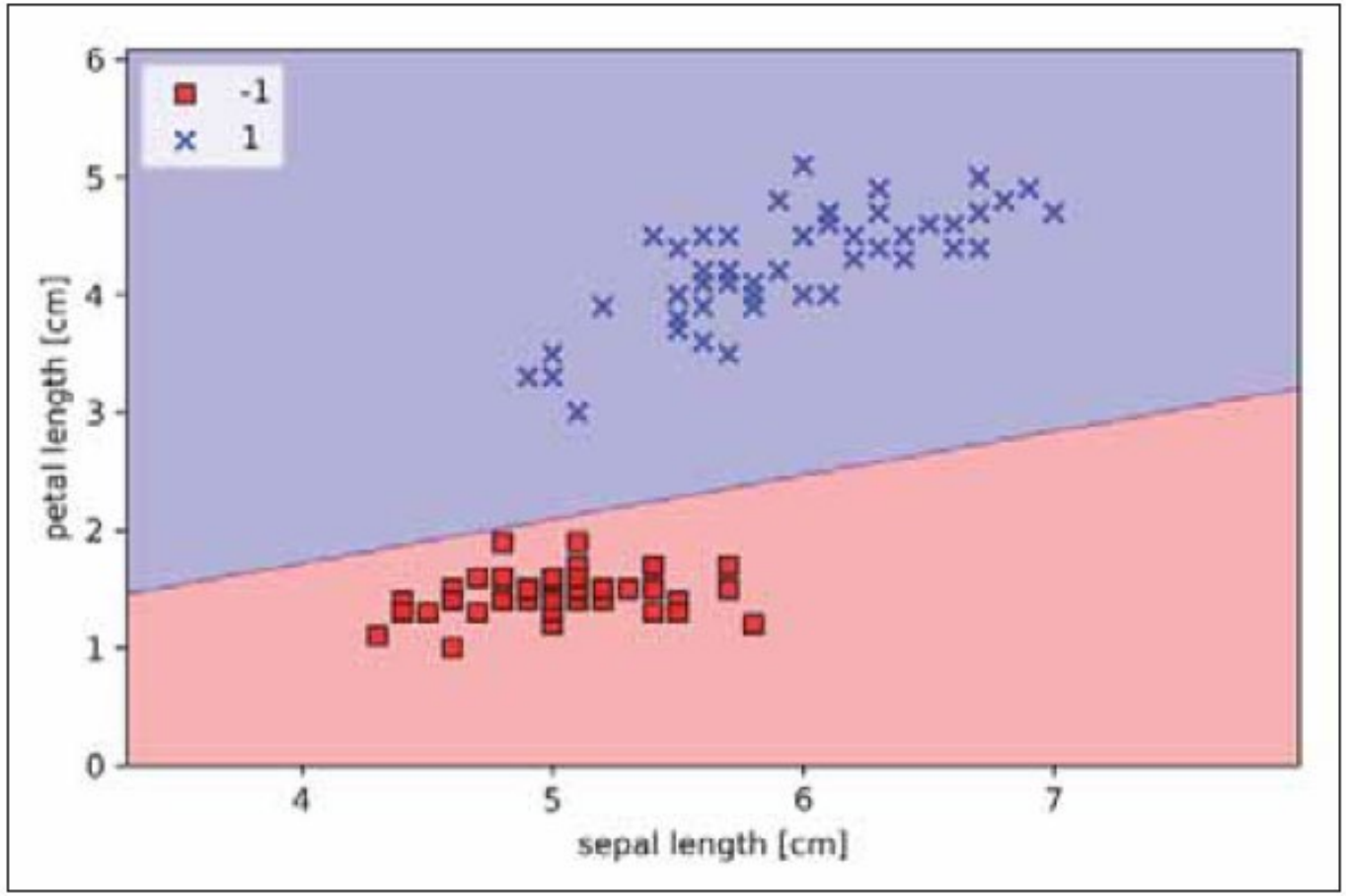
However, in the case of a wrong prediction, the weights are being pushed towards the direction of the positive or negative target class:

$$\Delta w_j = \eta\left(1 - -1\right)x_j^{(i)} = \eta\left(2\right)x_j^{(i)}$$

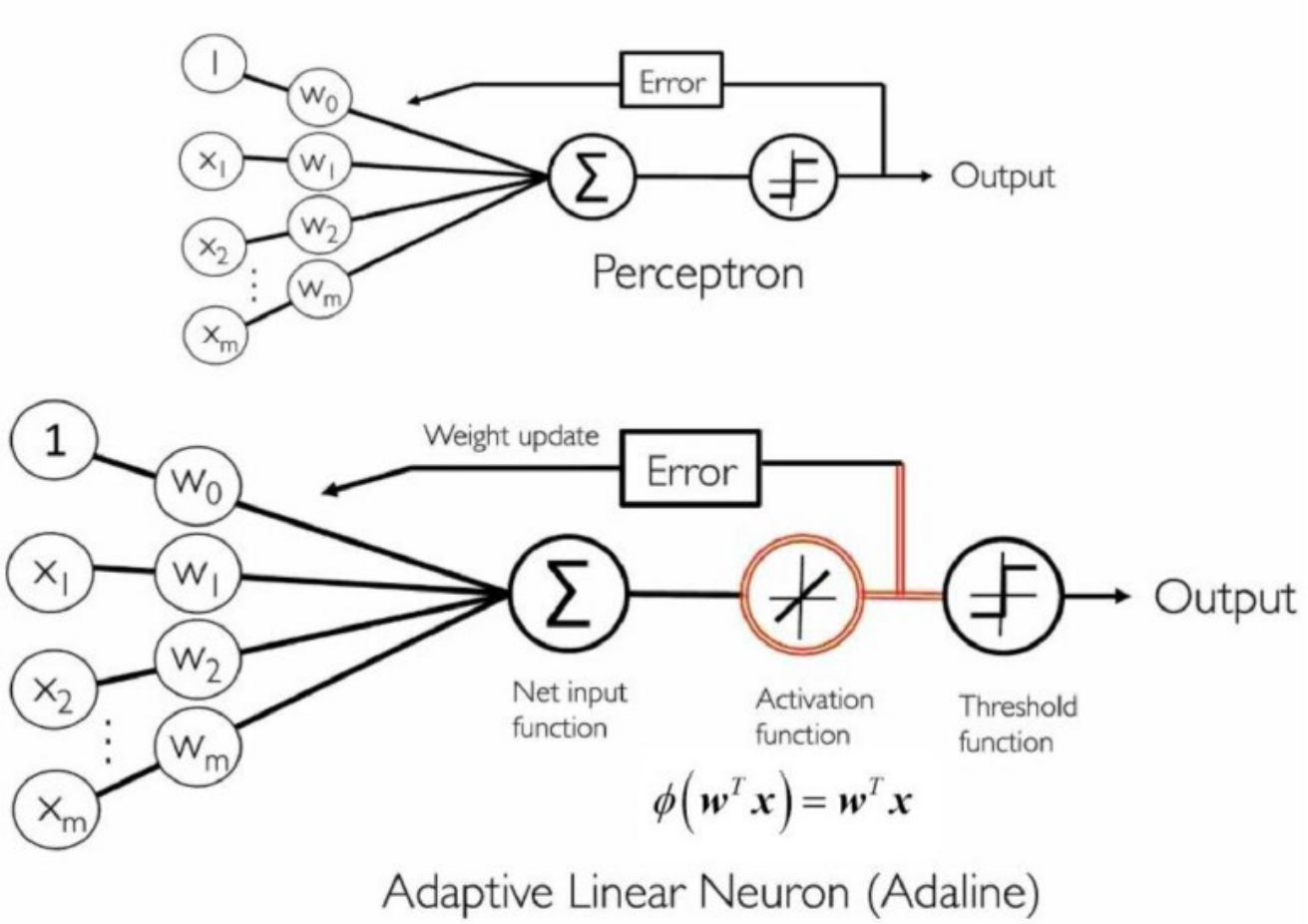
$$\Delta w_j = \eta\left(-1 - 1\right)x_j^{(i)} = \eta\left(-2\right)x_j^{(i)}$$



Perceptron for Classification



Adaline for Classification

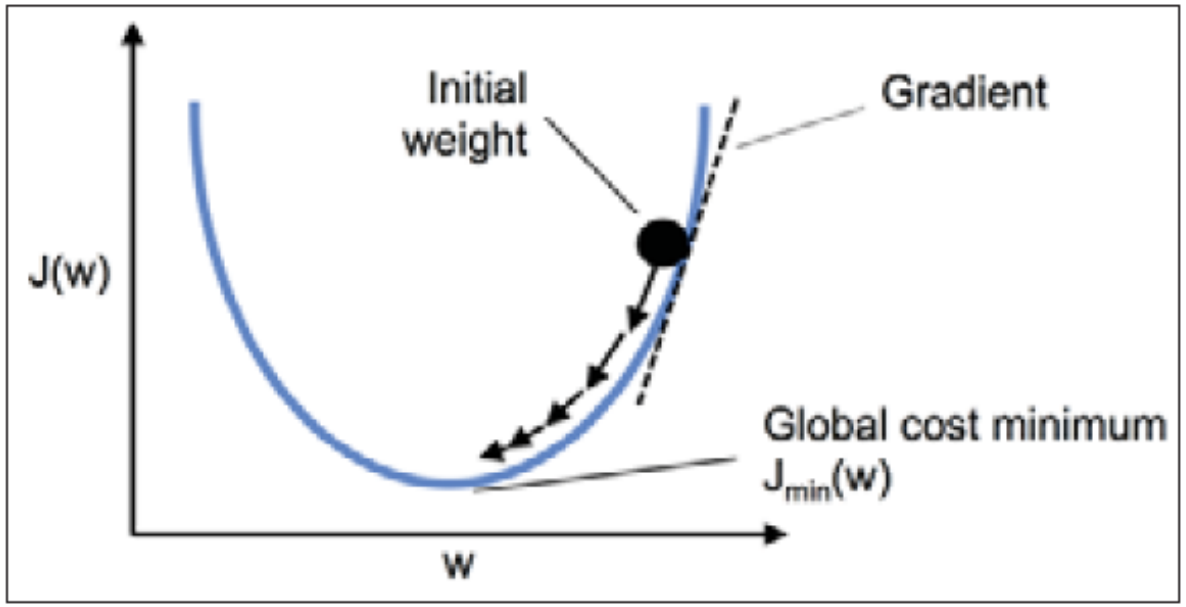


- Another type of single-layer neural network: ADAptive Linear Neuron (Adaline)
- Adaline was published in Oct. 1960 by Bernard Widrow and his doctoral student Tedd Hoff, after Frank Rosenblatt's perceptron algorithm

Adaline for Classification

- In the case of Adaline, we can define the cost function J to learn the weights as the sum of Squired Errors (SSE) between calculated outcome and the true class label:

$$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$



Weight Update

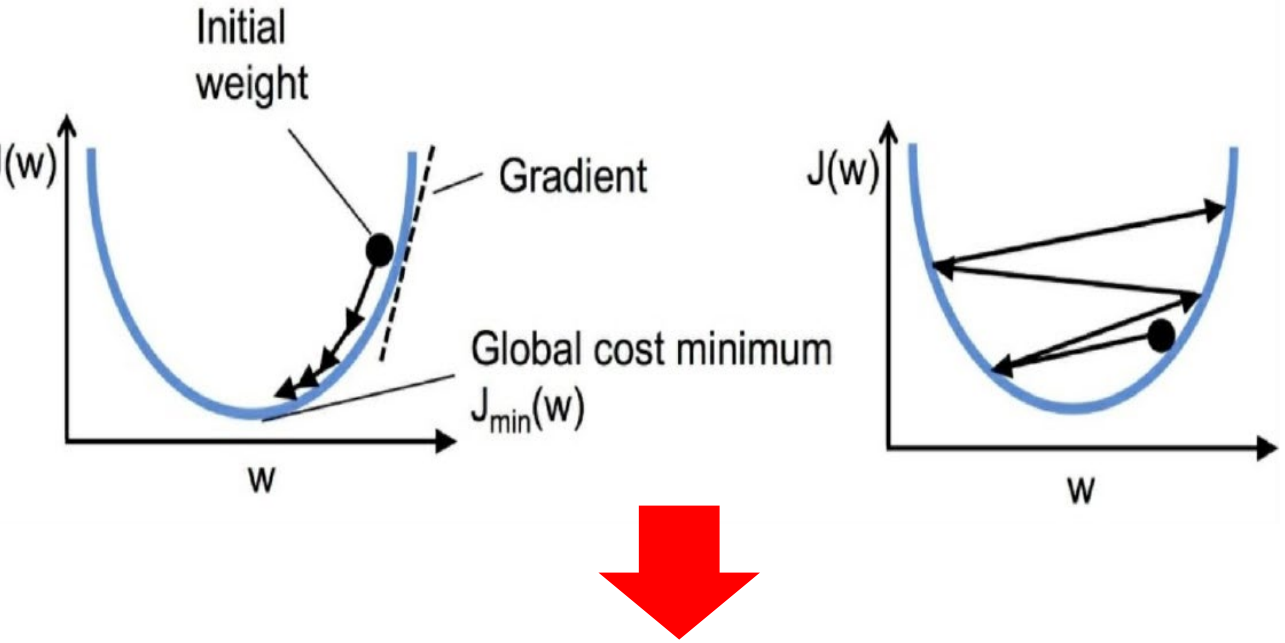
$$\boldsymbol{w} := \boldsymbol{w} + \Delta\boldsymbol{w}$$

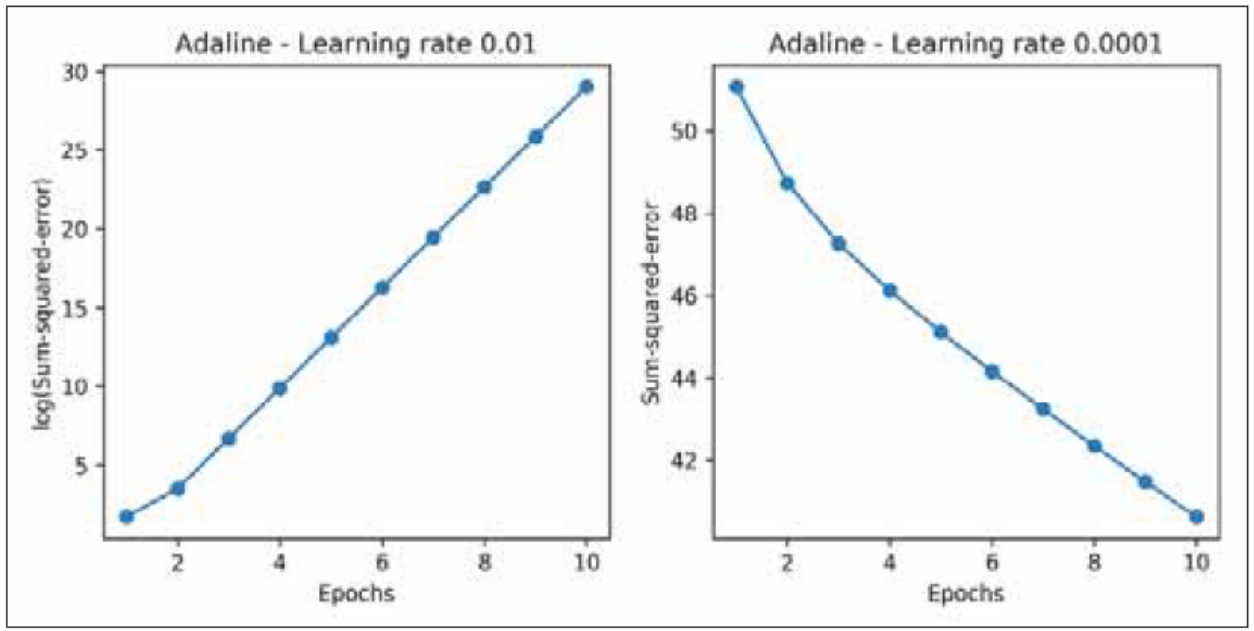
$$\Delta\boldsymbol{w} = -\eta\nabla J(\boldsymbol{w})$$

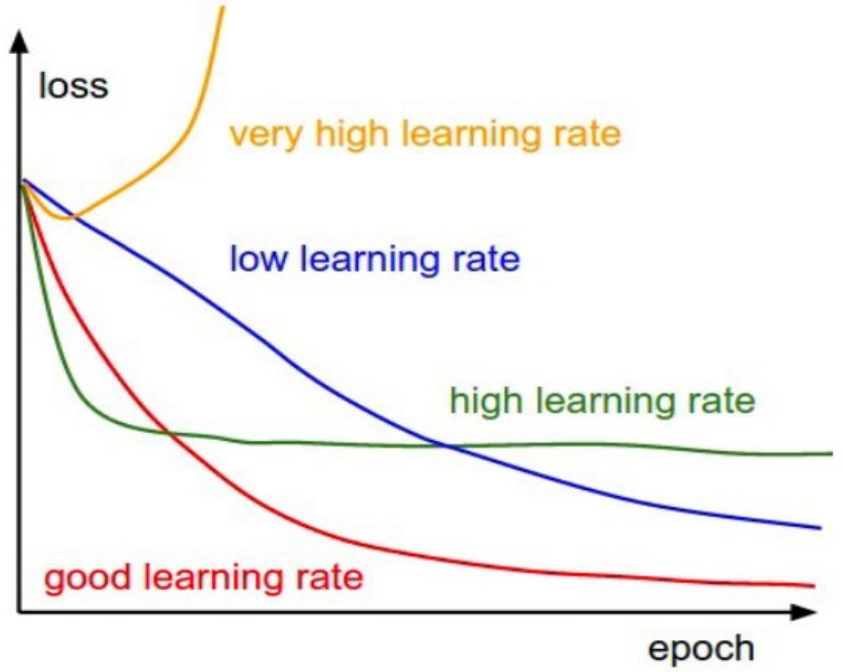
$$\Delta w_j = -\eta\frac{\partial J}{\partial w_j} = \eta\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)x_j^{(i)}$$

$$\Delta w_j = -\eta\frac{\partial J}{\partial w_j} = \eta\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)x_j^{(i)}$$

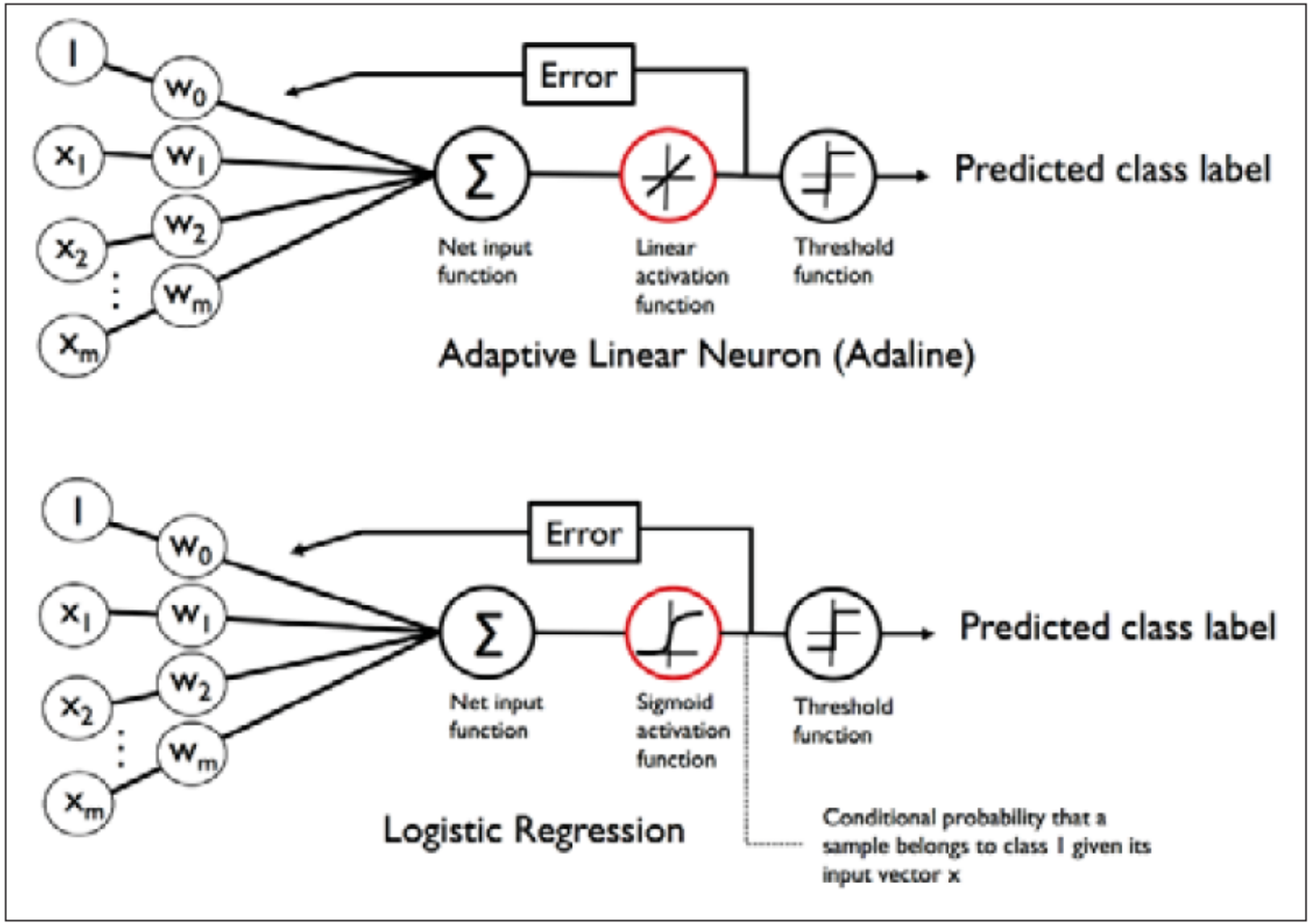
Adaline for Classification







Logistic Regression

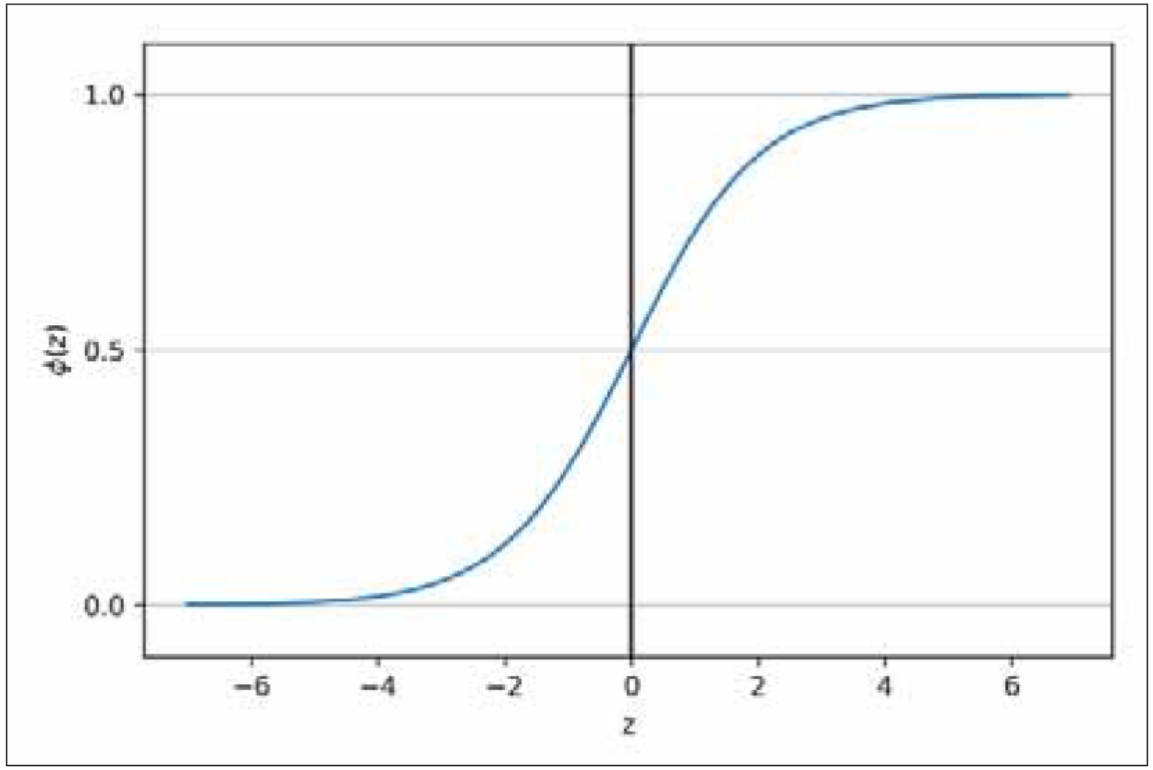


Logistic Regression

- The logistic sigmoid function, sometimes simply abbreviated as sigmoid function due to its characteristic S-shape:

$$\phi(z) = \frac{1}{1+e^{-z}}$$

$$z = \boldsymbol{w}^T \boldsymbol{x} = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m$$



$$\hat{y} = \begin{cases} 1 & \text{if } \phi(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Learning the weights of the logistic cost function

- The likelihood L that we want to maximize when we build a logistic regression model:

$$L(\boldsymbol{w}) = P(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{w}) = \prod_{i=1}^{n} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) = \prod_{i=1}^{n} \left(\phi\left(z^{(i)}\right)\right)^{y^{(i)}} \left(1 - \phi\left(z^{(i)}\right)\right)^{1-y^{(i)}}$$

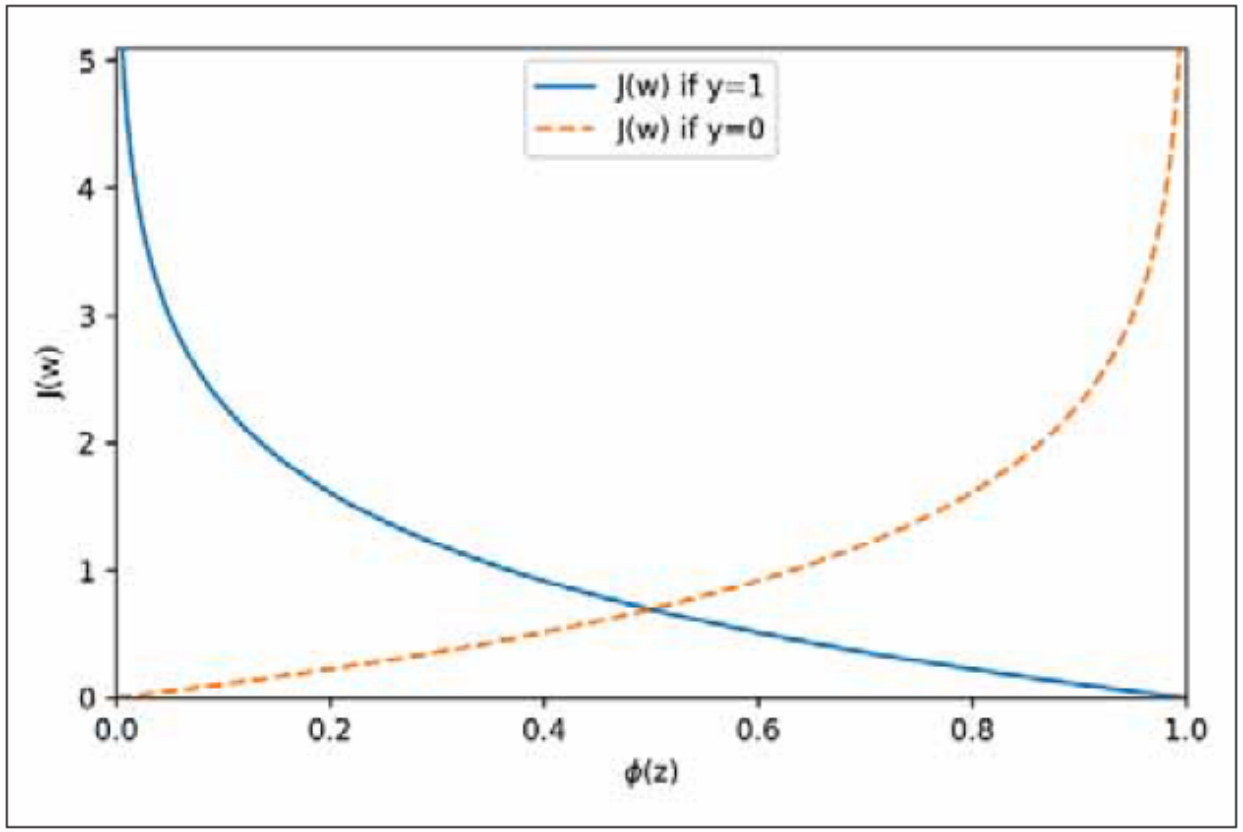
- The cost function:

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} \left[-y^{(i)} \log\left(\phi\left(z^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - \phi\left(z^{(i)}\right)\right) \right]$$

$$J\left(\phi(z), y; \boldsymbol{w}\right) = -y \log\left(\phi(z)\right) - (1-y) \log\left(1 - \phi(z)\right)$$

$$J\left(\phi(z), y; \boldsymbol{w}\right) = \begin{cases} -\log\left(\phi(z)\right) & \text{if } y = 1 \\ -\log\left(1 - \phi(z)\right) & \text{if } y = 0 \end{cases}$$

Learning the weights of the logistic cost function



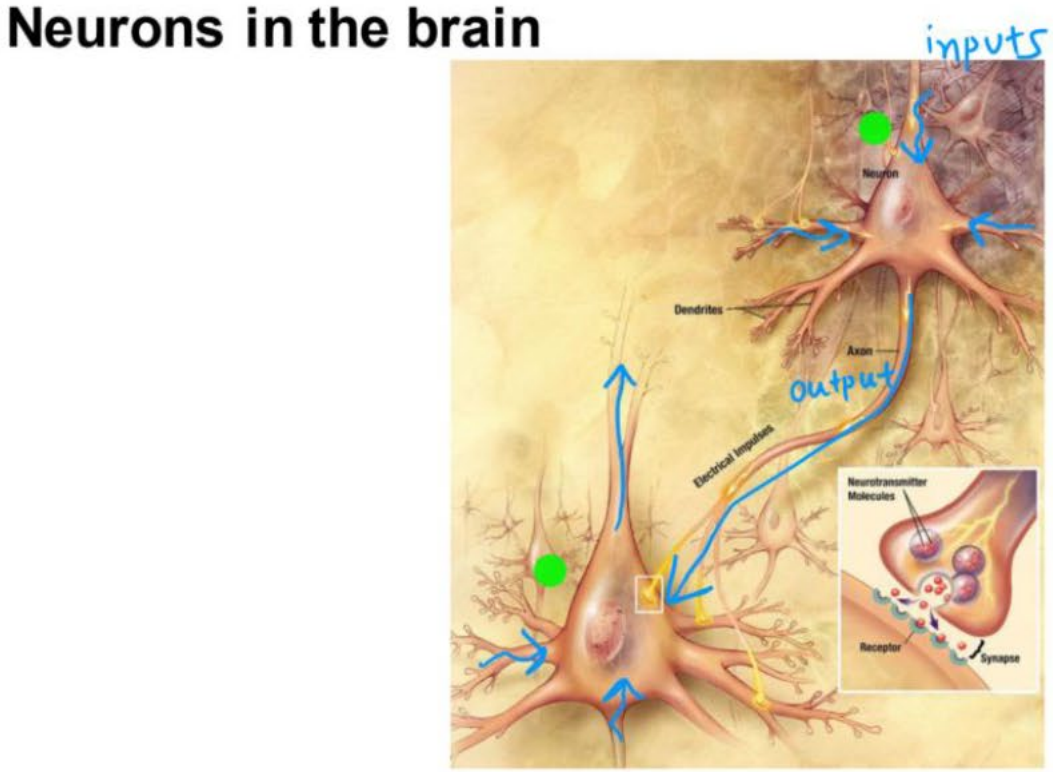
$$J\left(\phi(z), y; \boldsymbol{w}\right) = \begin{cases} -\log\left(\phi(z)\right) & if\ y = 1 \\ -\log\left(1-\phi(z)\right) & if\ y = 0 \end{cases}$$

Updating Weight:

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_{i=1}^{n} \left(y^{(i)} - \phi\left(z^{(i)}\right) \right) x_j^{(i)}$$

$$\boldsymbol{w} := \boldsymbol{w} + \Delta \boldsymbol{w},\ \Delta \boldsymbol{w} = -\eta \nabla J\left(\boldsymbol{w}\right)$$

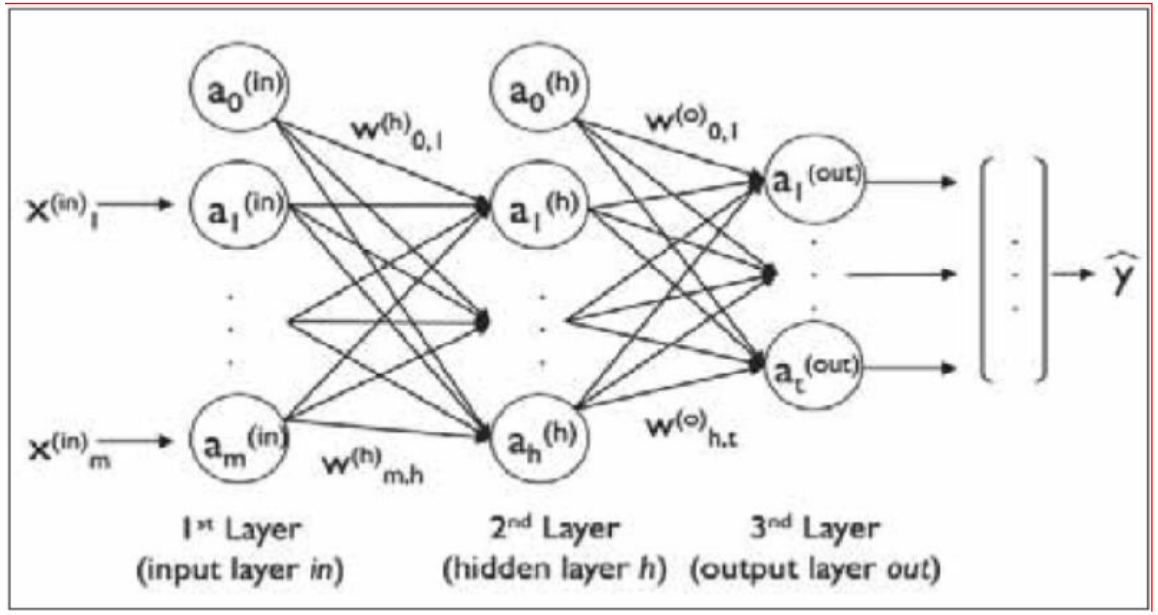
Deep Neural Networks

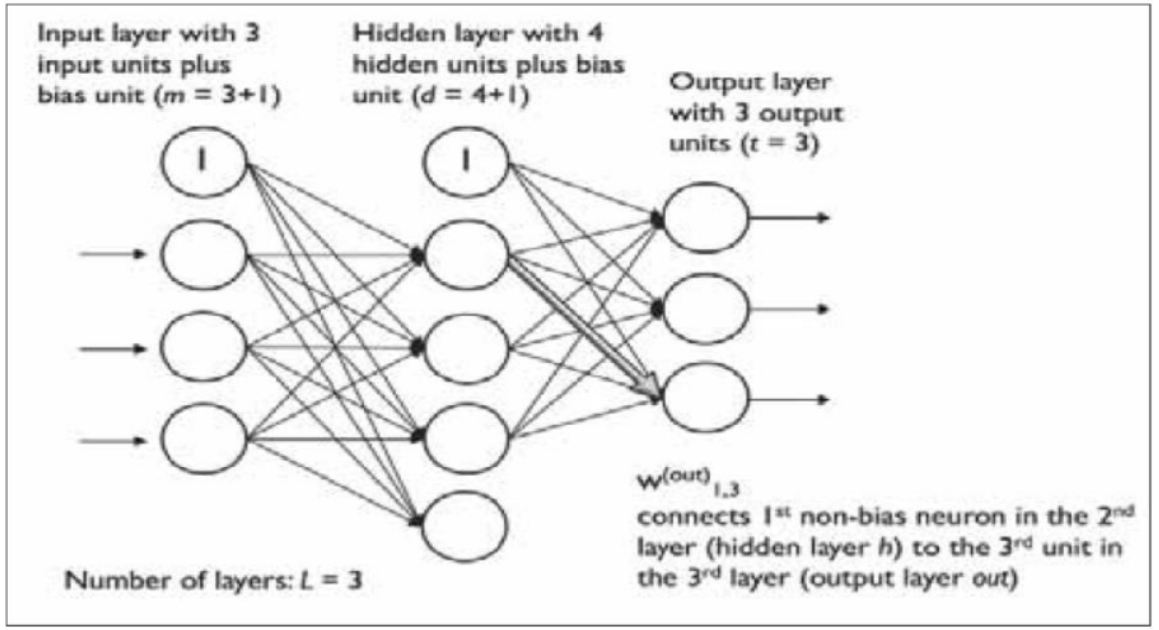


[Credit: US National Institutes of Health, National Institute on Aging]

Multilayer Neural Network Architecture

- Amultilayerfeedforwardneuralnetwork;thisspecialtypeof*fullyconnected*networkisalsocalled**MultilayerPerceptron(MLP)**
- The MLP depicted in the preceding figure has one input layer, one hidden layer, and output layer
- The units in the hidden layer are fully connected to the input layer, and the output layer is fully connected to the hidden layer
- If such a network has more than one hidden layer, we also call it a deep artificial neural network





Activating Neural Network via Forward Propagation

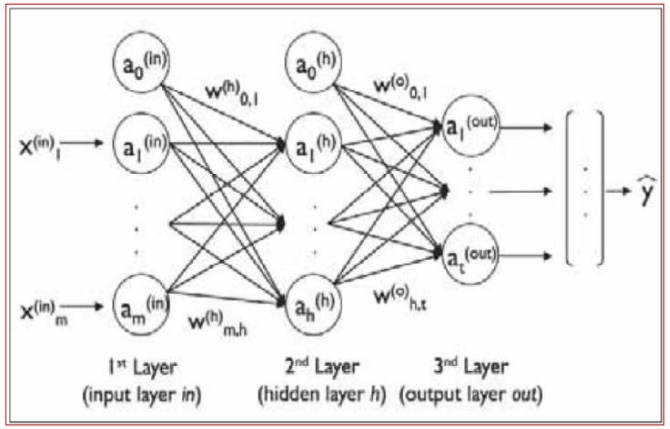
- The MLP learning procedure in three simple steps:
- Starting at the input layer, we forward propagate the patterns of the training data through the network to generate an output
- Based on the network's output, we calculate the error that we want to minimize using a cost function that we will describe later
- We back propagate the error, find its derivative with respect to each weight in the network, and update the model

➢ We first calculate the activation unit of the hidden layer a_1^h as follows:

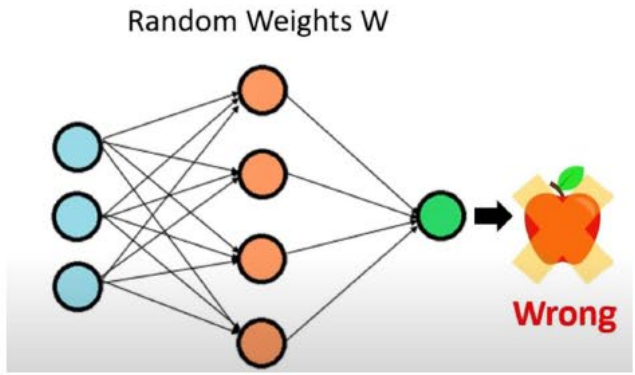
$$z_1^{(h)} = a_0^{(in)} w_{0,1}^{(h)} + a_1^{(in)} w_{1,1}^{(h)} + \cdots + a_m^{(in)} w_{m,1}^{(h)}$$

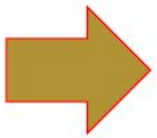
$$a_1^{(h)} = \phi\left(z_1^{(h)}\right)$$

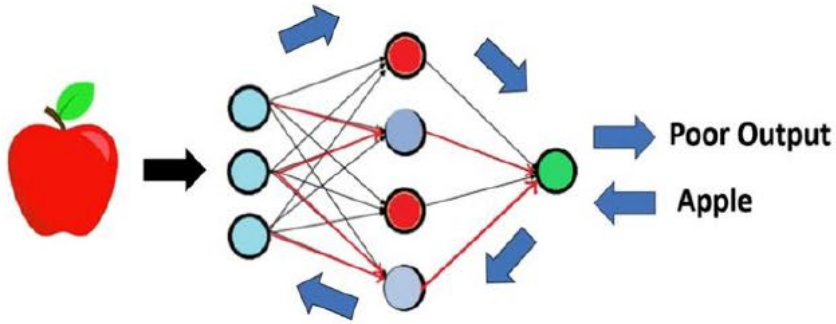
$$\phi(z) = \frac{1}{1+e^{-z}}$$



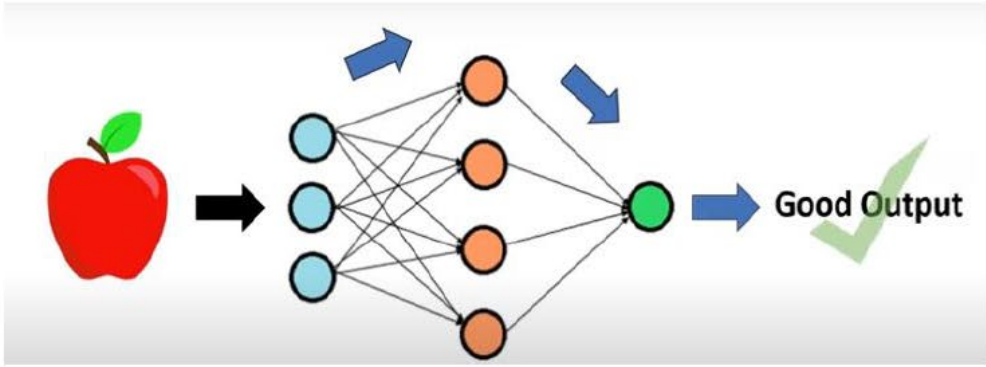
Activating Neural Network via Forward Propagation







It means our model has an error. Then, we can we do?

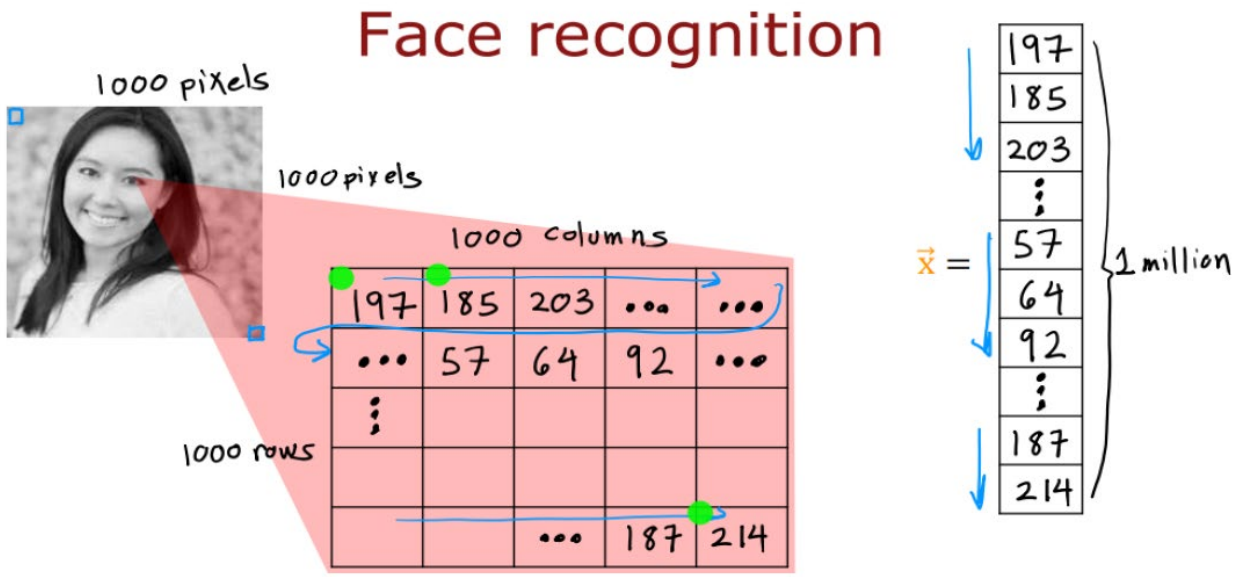


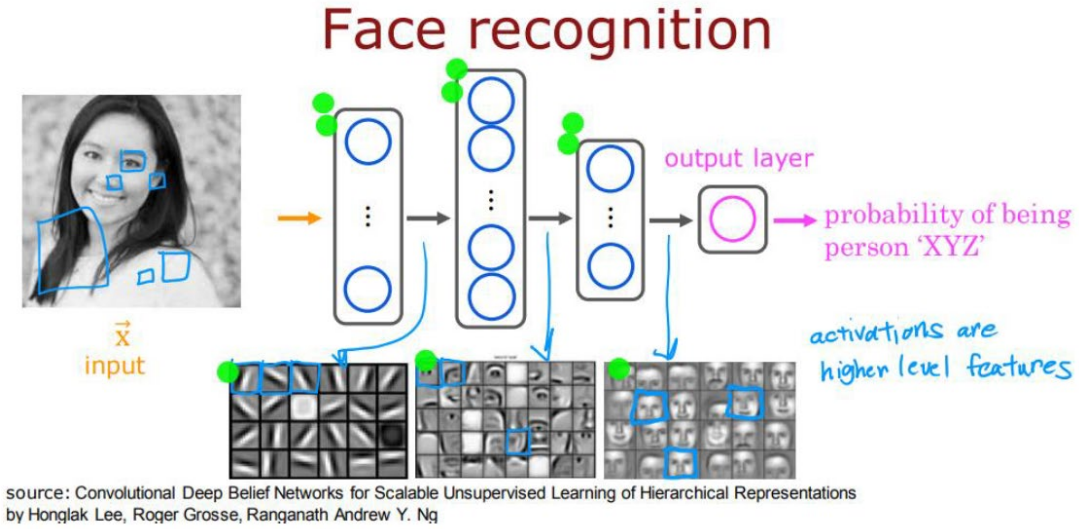
AALBORG UNIVERSITY

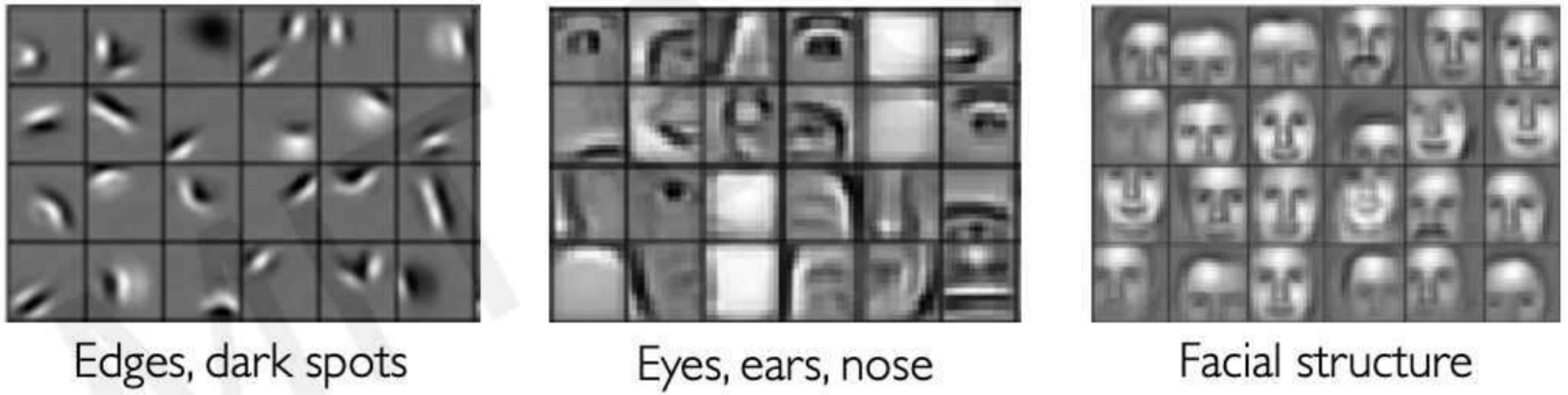
PAGE 9

Image source: https://www.youtube.com/watch?v=Tb23YtZ92AE

Example of MLP

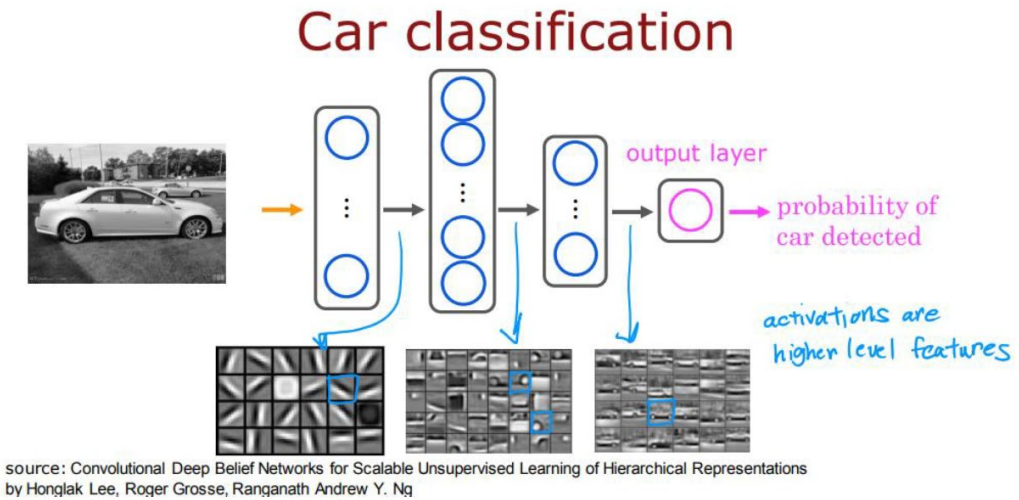




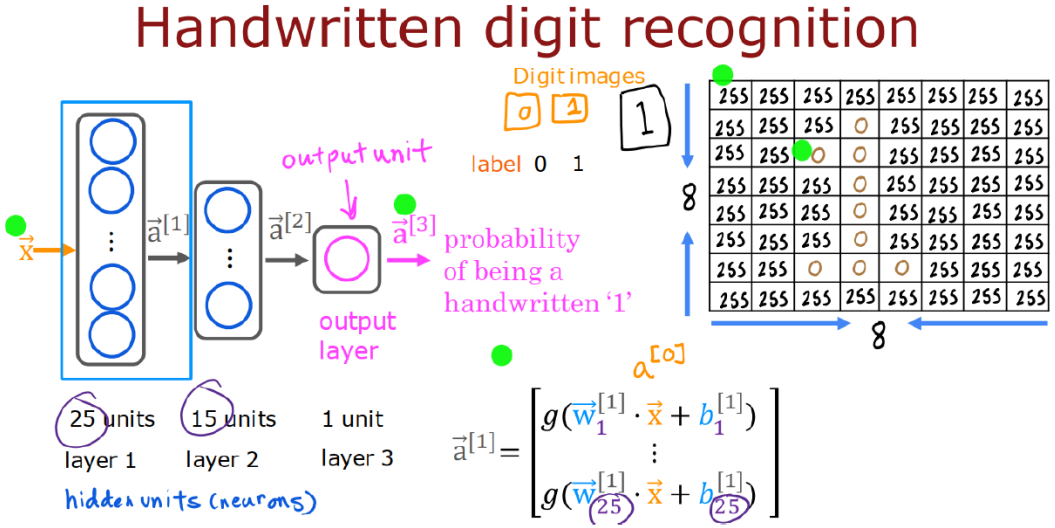


Source: Lee, Honglak, et al. "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations." *Proceedings of the 26th annual international conference on machine learning*. 2009.

Example of MLP



source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng



Parallelizing Neural Network Training with TensorFlow

- TensorFlow is a scalable and multiplatform programming interface for implementing and running machine learning algorithms, including convenience wrappers for deep learning
- TensorFlow was developed by the researchers and engineers of the Google Brain team
- TensorFlow was initially built for only internal use at Google, but it was subsequently released in November 2015 under a permissive open source license
- To improve the performance of training machine learning models, TensorFlow allows execution on both CPUs and GPUs. However, its greatest performance capabilities can be discovered when using GPUs
- TensorFlow supports CUDA-enabled GPUs officially

Parallelizing Neural Network Training with TensorFlow

- Pip install tensorflow
- Pip install tensorflow-gpu
- TensorFlow is built around a computation graph composed of a set of nodes. Each node represents an operation that may have zero or more input or output. The values the flow through the edges of the computations graph are called **tensors**

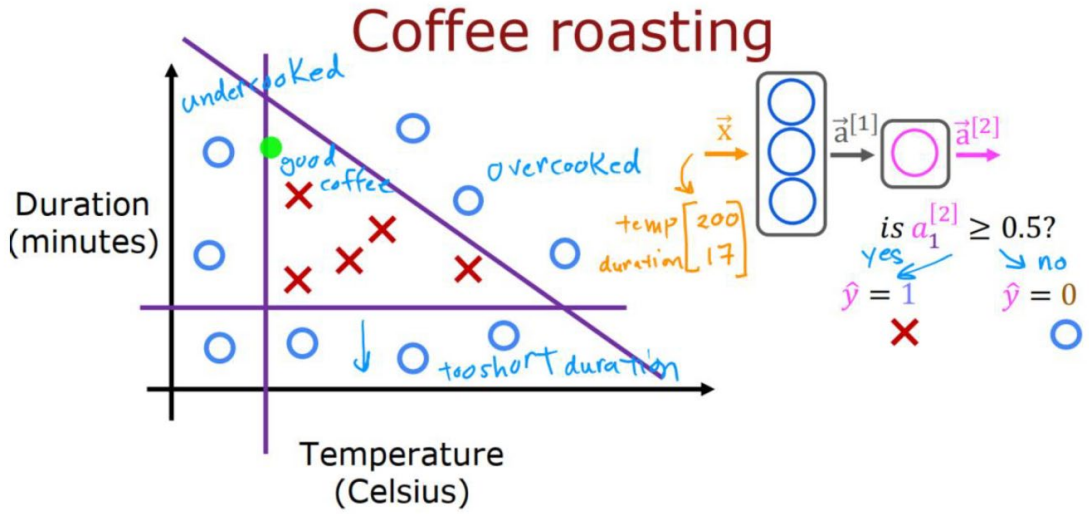
$$z = w \times x + b$$

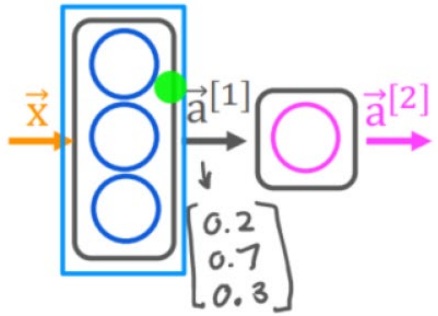
```
import tensorflow as tf

## create a graph
g = tf.Graph()
with g.as_default():
    x = tf.placeholder(dtype=tf.float32,
                       shape=(None), name='x')
    w = tf.Variable(2.0, name='weight')
    b = tf.Variable(0.7, name='bias')

    z = w*x + b

    init = tf.global_variables_initializer()
## create a session and pass in graph g
with tf.Session(graph=g) as sess:
    ## initialize w and b:
    sess.run(init)
    ## evaluate z:
    for t in [1.0, 0.6, -1.8]:
        print('x=%4.1f --> z=%4.1f'%(
              t, sess.run(z, feed_dict={x:t})))
```

After executing the previous code, you should see the following output:

```
x= 1.0 --> z= 2.7
x= 0.6 --> z= 1.9
x=-1.8 --> z=-2.9
```

Parallelizing Neural Network Training with TensorFlow



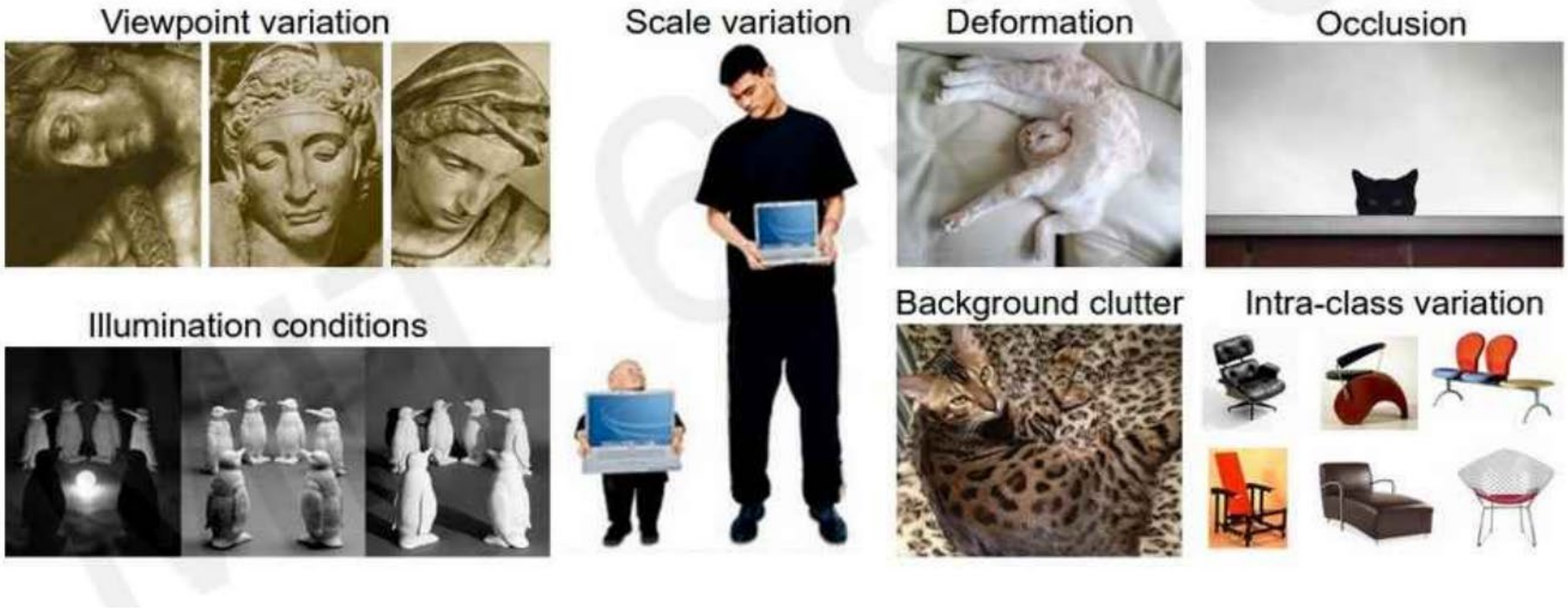



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

Disadvantages of Using MLP

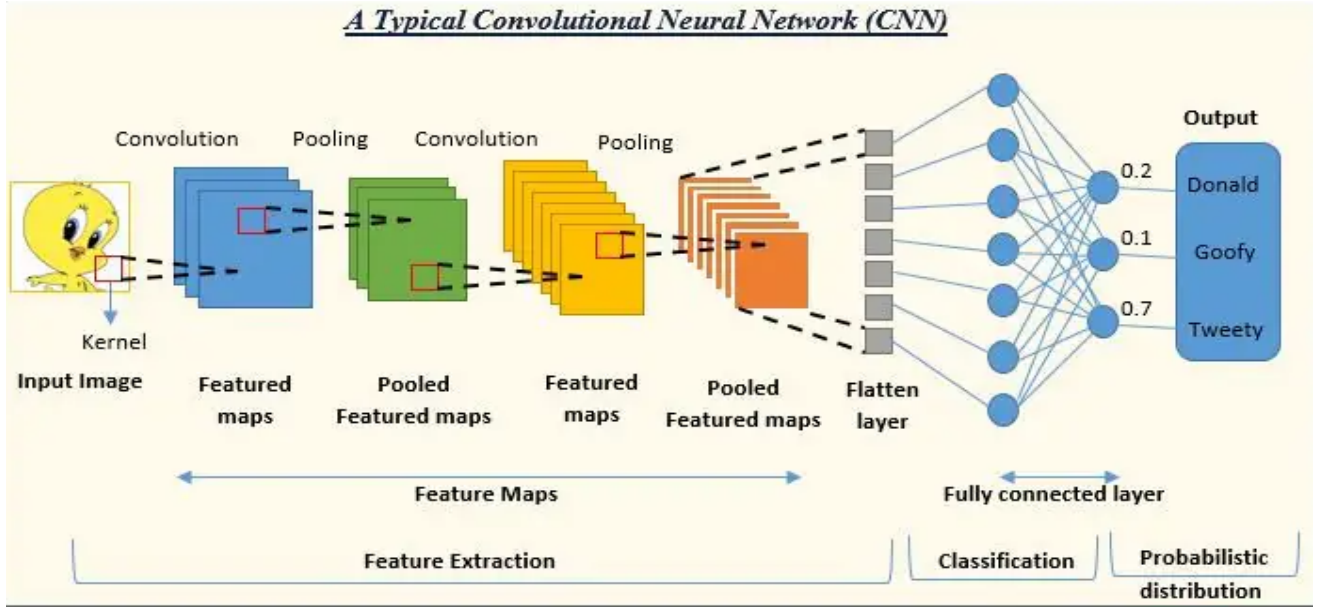
- **Too many parameters**
  - Fully connected layers create millions of weights for large images → high memory usage & slow training
- **Ignores spatial structure**
  - MLP flattens images into vectors → loses 2D relationships between neighboring pixels
- **Sensitive to position changes**
  - A small shift in an object changes many input values → poor translation handling
- **Prone to overfitting on large images**
  - A large number of parameters increases the risk of memorizing instead of learning patterns

Convolutional Neural Network (CNN)

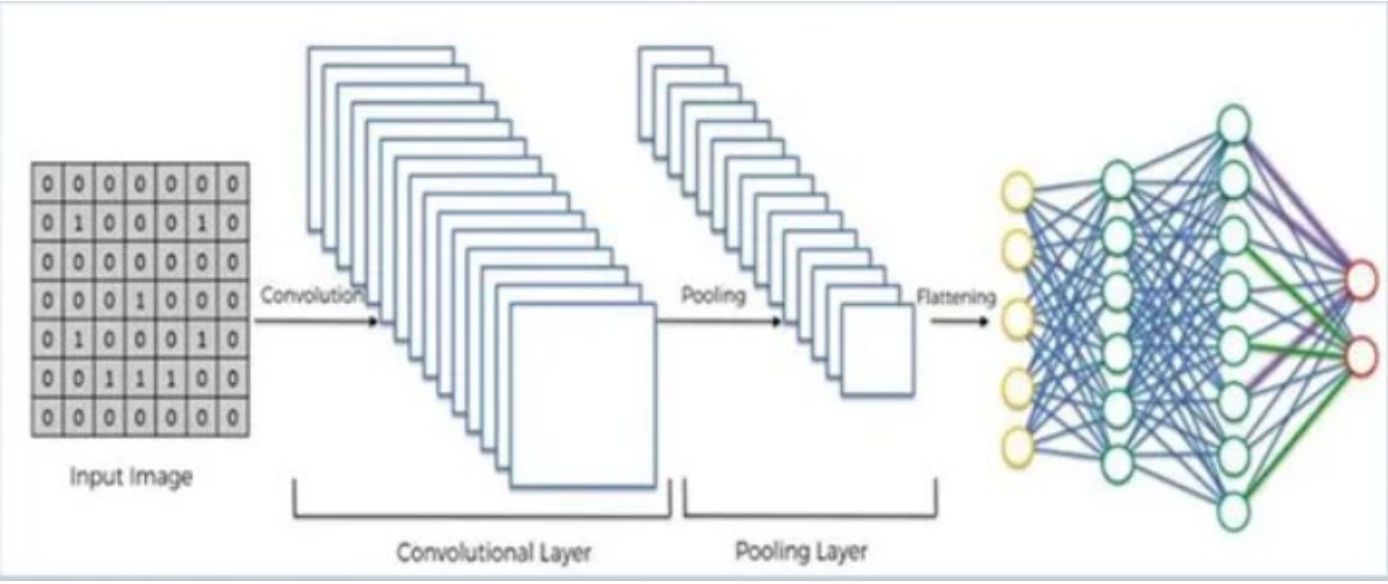


Source: https://introtodeeplearning.com/

Convolutional Neural Network (CNN)



- ✓ More aggressive downsampling
- ✓ Smaller FC
- ✓ Safer for small datasets



- ✓ Better feature learning
- ✓ More expressive
- ✓ May overfit if FC is large

Source:https: //glasswing.vc/blog/ai-atlas-16-convolutional-neural-networks-cnns/

Convolutional Neural Network (CNN)

Convolution Layer

- Convolutional layers are the building blocks of **Convolutional Neural Networks (CNNs)**, used primarily for analyzing visual data
- Extract features (edges, textures, shapes) from images while preserving spatial relationships
- Instead of connecting every neuron to every input (like fully connected layers), convolutional layers use **small filters/kernels** that slide over the input

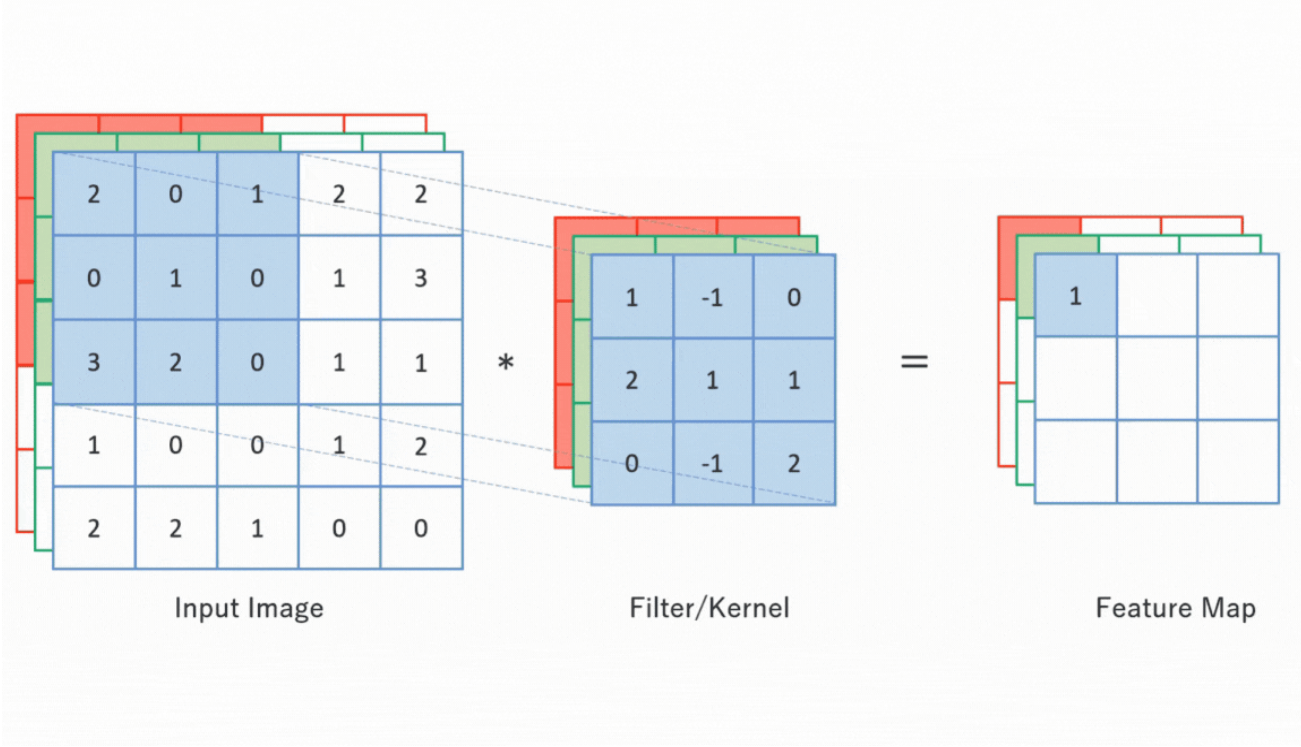


Image Source: https://medium.com/@alejandro.itoaramendia/convolutional-neural-networks-cnns-a-complete-guide-a803534a1930

Convolutional Neural Network (CNN)

Convolution Layer

- **Filter/Kernel:** Small matrix (e.g., 3x3 or 5x5) that detects patterns. There will be multiple filters to detect multiple features
- **Stride:** Step size of the filter as it moves across the input
- Smaller filters + strid 1 -> more detail preserved, Larger filters / stride >1 -> faster downsampling, fewer computations
- **Padding:** Adding borders to input to control output size
- **Feature Map / Activation Map:** Result of applying filter to input.

Padding

- valid padding (no padding): No extra pixels are added; convolution is applied only to valid positions of the input

Input size, kernel/filter size

$$O = \frac{I - K}{S} + 1$$

Output size, Stride

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

⟹

13	14	15
18	19	20
23	24	25

Image Source: https://www.geeksforgeeks.org/deep-learning/what-is-the-difference-between-same-and-valid-padding-in-tf-nn-max_pool-of-tensorflow/

Convolutional Neural Network (CNN)

Padding

- Same Padding (Zero Padding): Add zeros around the input so that the **output size equals input size**

Kernel size

$$P = \frac{(K-1)}{2}$$

Padding

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0

⟹

7	8	9	10	10
12	13	14	15	15
17	18	19	20	20
22	23	24	25	25
22	23	24	25	25

Padding

- Full Padding: Add padding so that every element of the kernel can slide over all positions of the input, including corners (Less Common)

Input size Kernel size

$$O = I + K + 1$$

Output size

$$P = K - 1$$

Image Source: https://www.geeksforgeeks.org/deep-learning/what-is-the-difference-between-same-and-valid-padding-in-tf-nn-max_pool-of-tensorflow/

Convolutional Neural Network (CNN)

How Convolution Works

- Place kernel on input
- Multiply overlapping values element-wise
- Sum the results → one value in the feature map
- Slide kernel (stride) and repeat

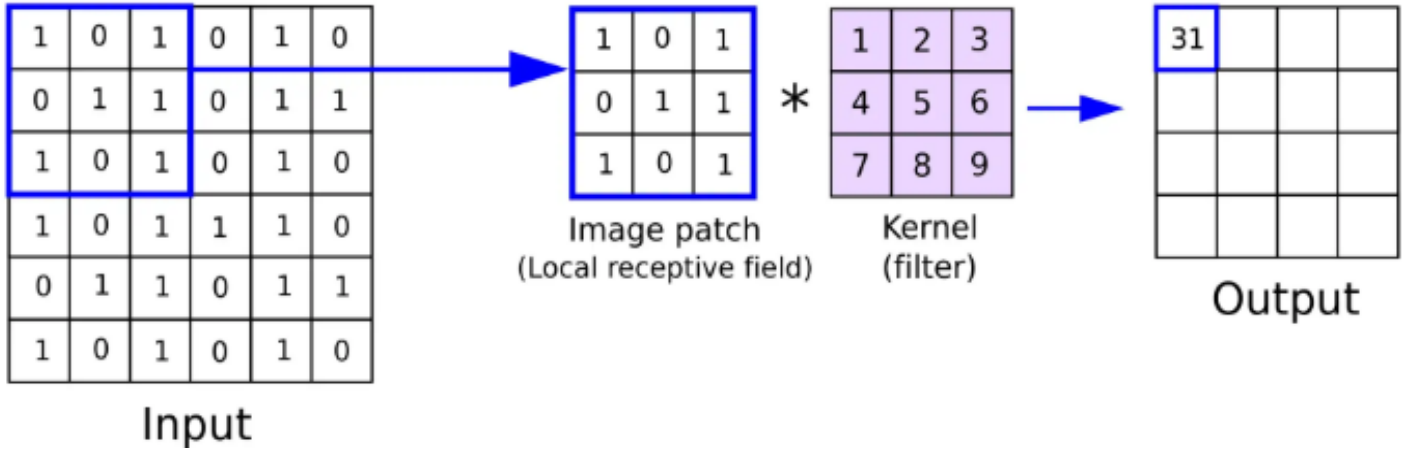


Image Source: https://www.superannotate.com/blog/guide-to-convolutional-neural-networks

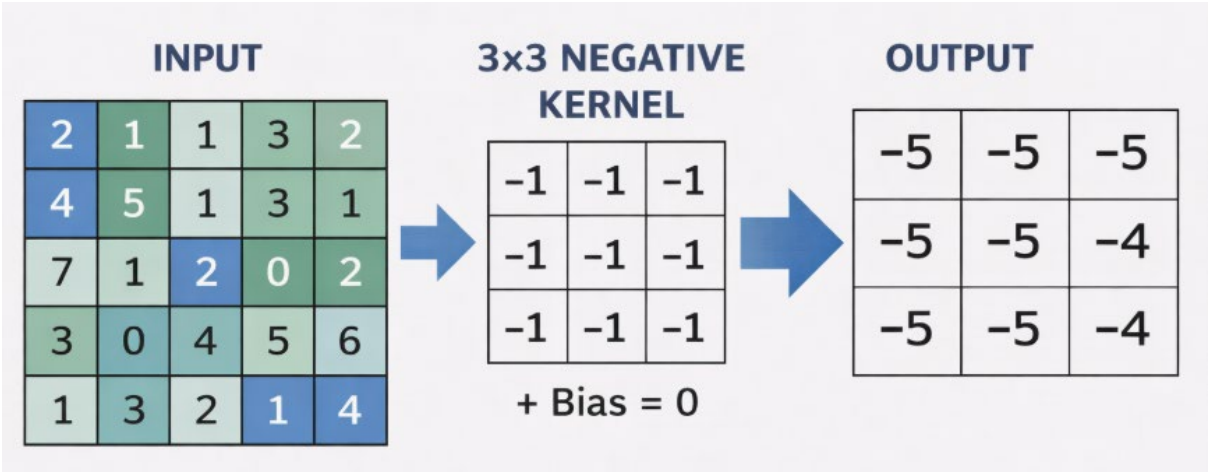
Convolutional Neural Network (CNN)

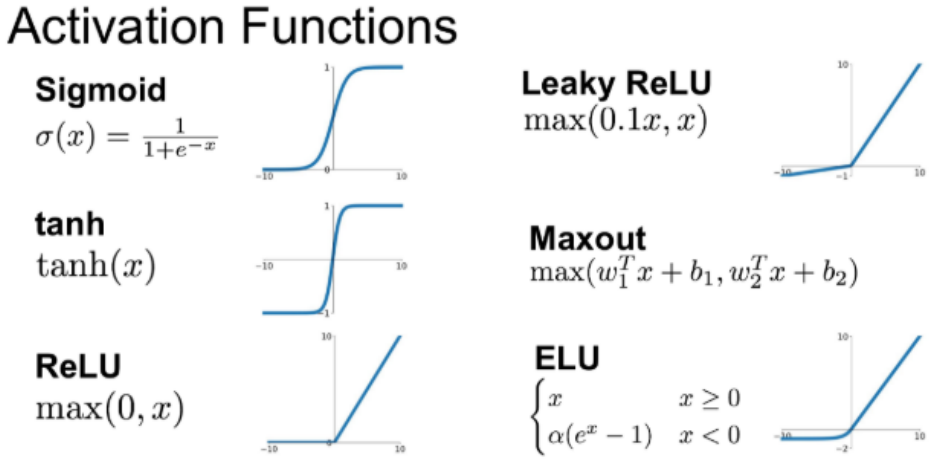
Convolution Layer

- **Number of Filters:** More filters → more features detected
- **Activation Function:** Usually ReLU (Retified Linear Unit) applied after convolution

$$f(x) = \max(0, x)$$

- Can Only separate data that is linearly separatable, but cannot learn hierarchical features (edges, shapes, object parts) without apply ReLU
- In ReLU function, it can happen that the neuron always outputs “0” (dying ReLU)





Convolutional Neural Network (CNN)

Pooling Layer

- **Pooling Layers:** Often follow convolution (e.g., MaxPooling, MinPooling, AvgPooling) to reduce dimensions
- The common filter size 2x2 or 3x3. The stride is same with filter size

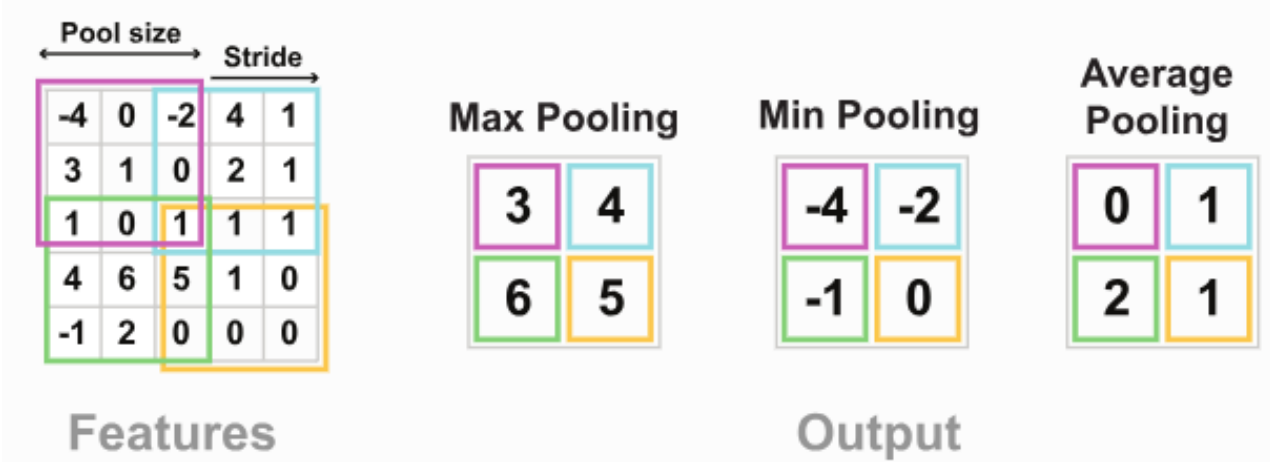


Image Source: https://epynn.net/Pooling.html

Convolutional Neural Network (CNN)

Fully Connected Layers

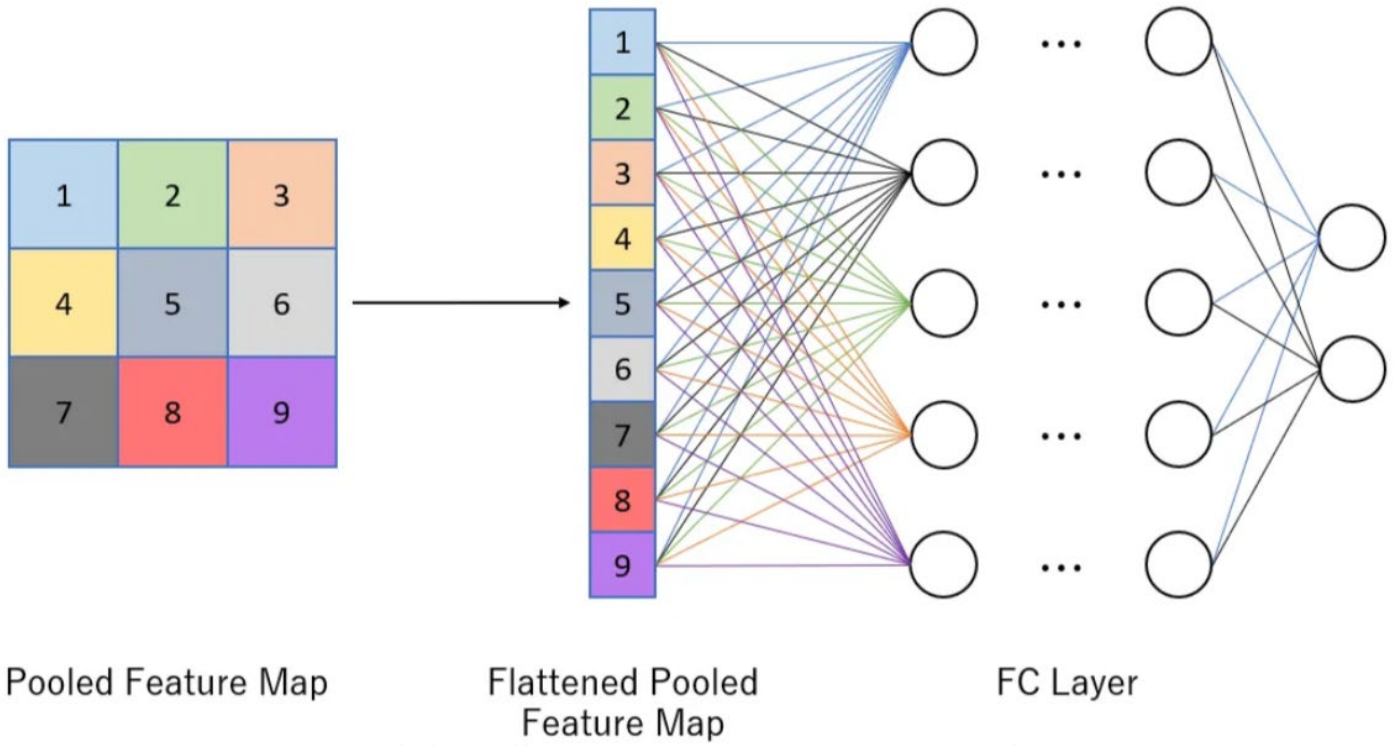
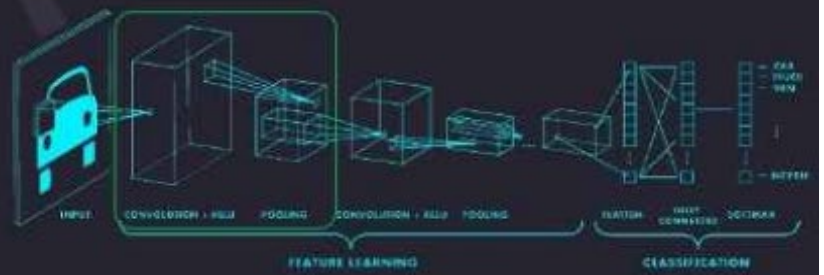


Image Source: https://medium.com/@alejandro.itoaramendia/convolutional-neural-networks-cnns-a-complete-guide-a803534a1930

Convolutional Neural Network (CNN)

Putting it all together: CNN in TensorFlow

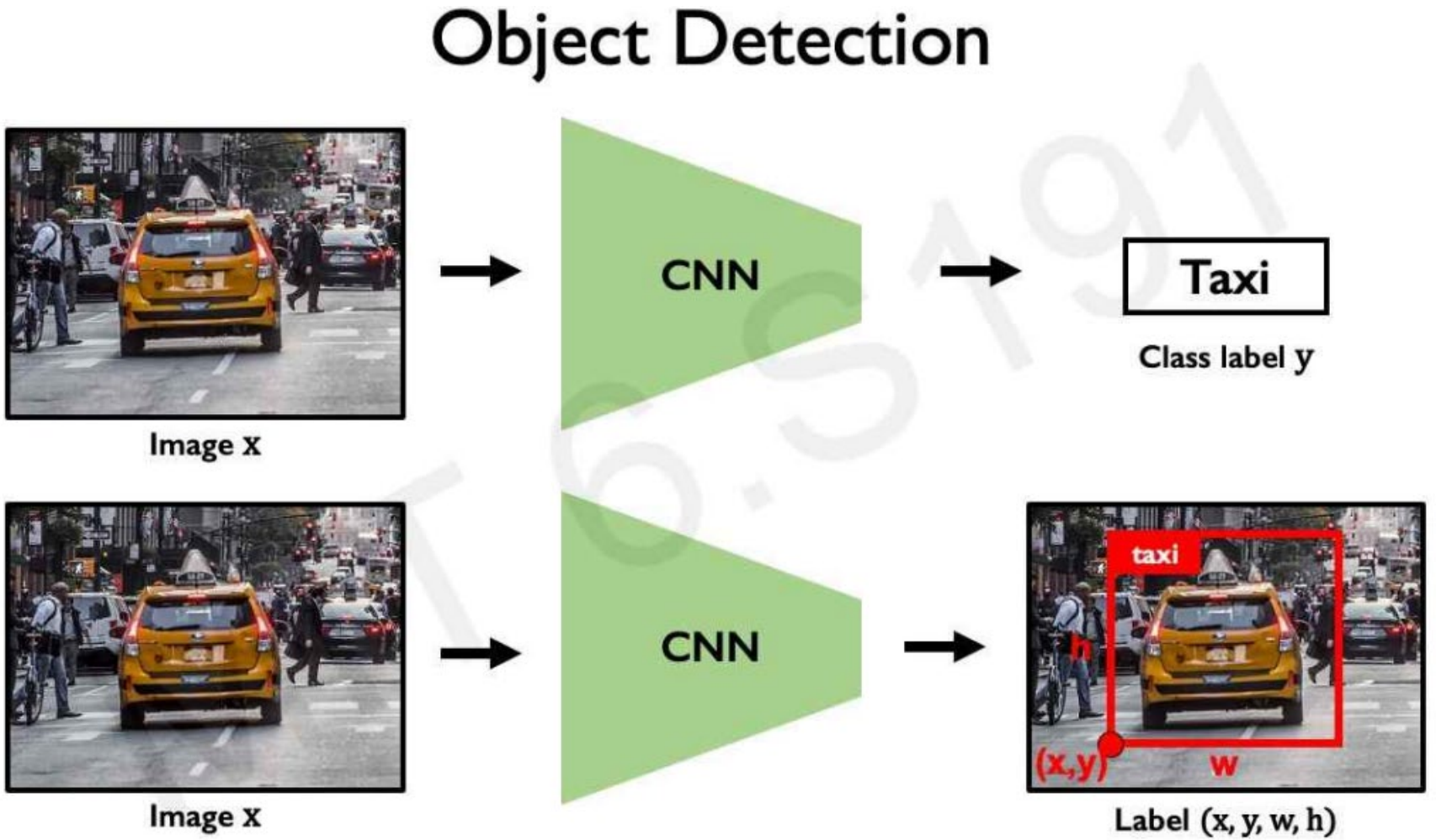
```
import tensorflow as tf

def generate_model():
    model = tf.keras.Sequential([
        # first convolutional layer
        tf.keras.layers.Conv2D(32, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # second convolutional layer
        tf.keras.layers.Conv2D(64, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # fully connected classifier
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1024, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')   # 10 outputs
    ])
    return model
```



MIT 6.S191

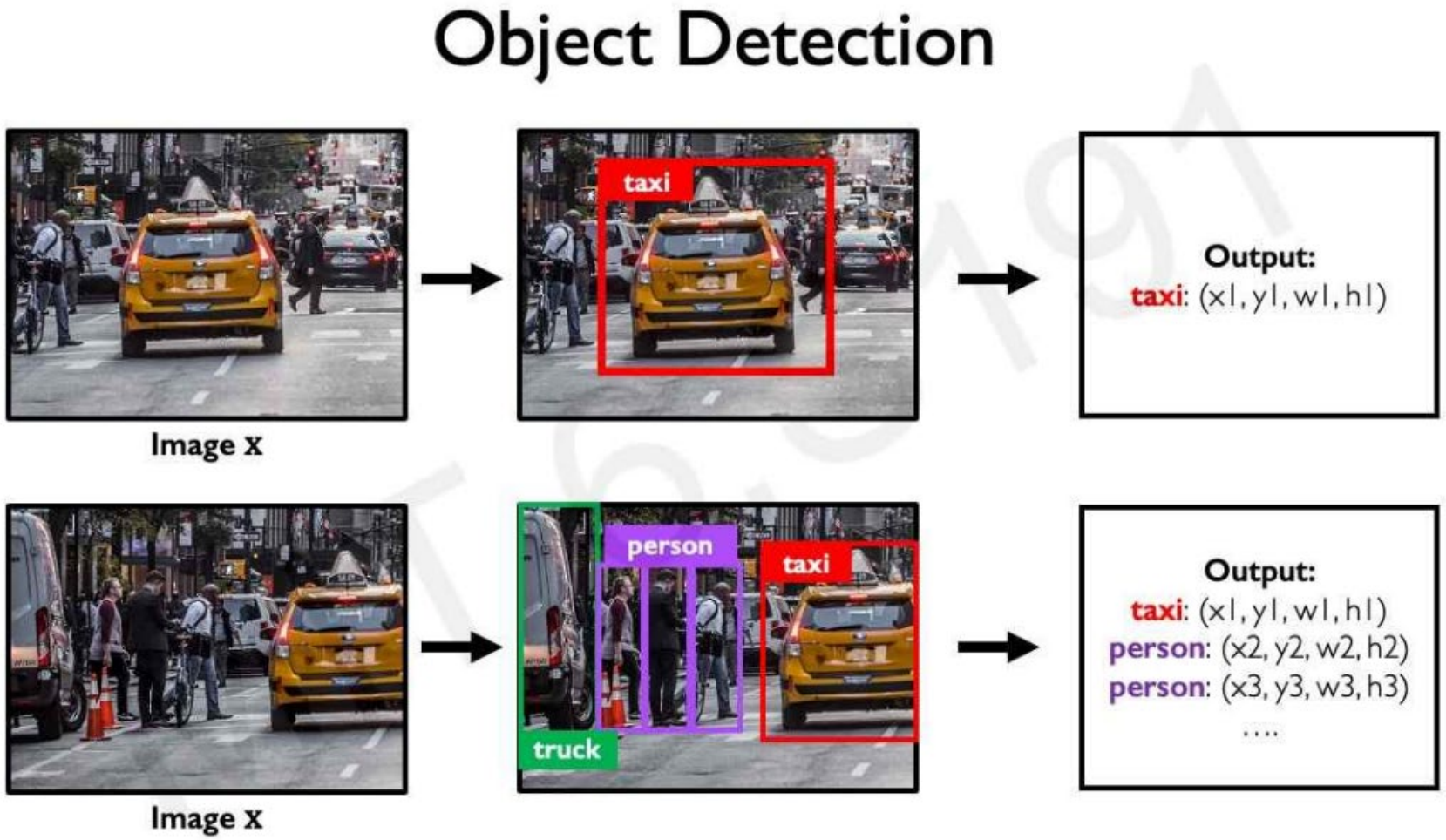
© Alexander Amini and Ava Amini

Applications of CNN



© Alexander Amini and Ava Amini

Convolutional Neural Network (CNN)



© Alexander Amini and Ava Amini

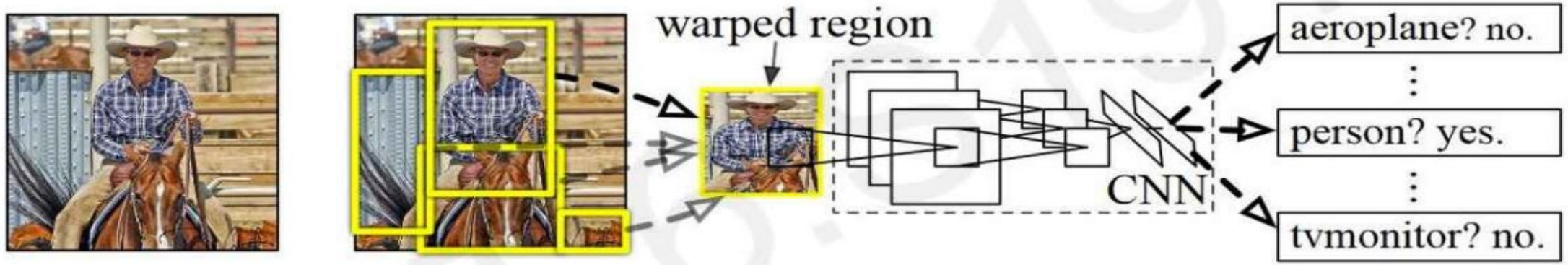
Convolutional Neural Network (CNN)

Problem: Way too many inputs! This results in too many scales, positions, sizes!

© Alexander Amini and Ava Amini

Convolutional Neural Network (CNN)

R-CNN algorithm: Find regions that we think have objects. Use CNN to classify.



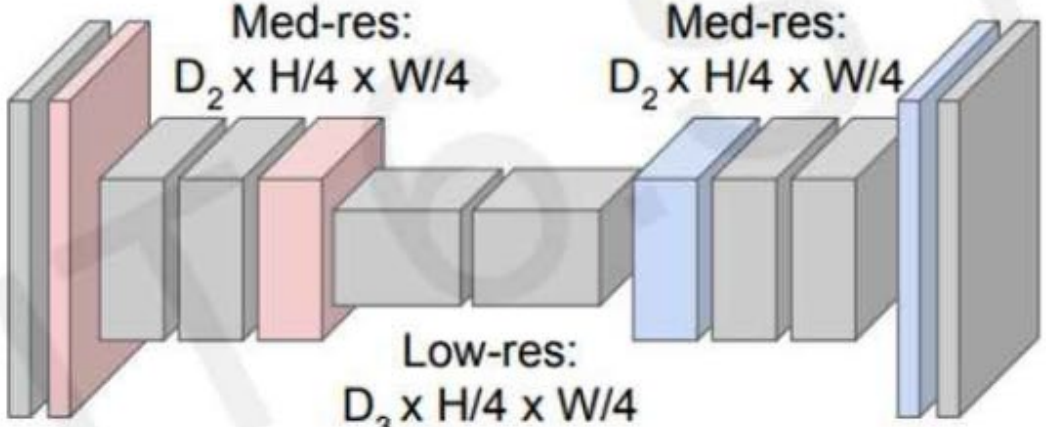
Problems: 1) Slow! Many regions; time intensive inference.
2) Brittle! Manually defined region proposals.

© Alexander Amini and Ava Amini

Semantic Segmentation: Fully Convolutional Networks

FCN: Fully Convolutional Network.
Network designed with all convolutional layers,
with **downsampling** and **upsampling** operations

Input:
3 x H x W



High-res:
D_1 x H/2 x W/2

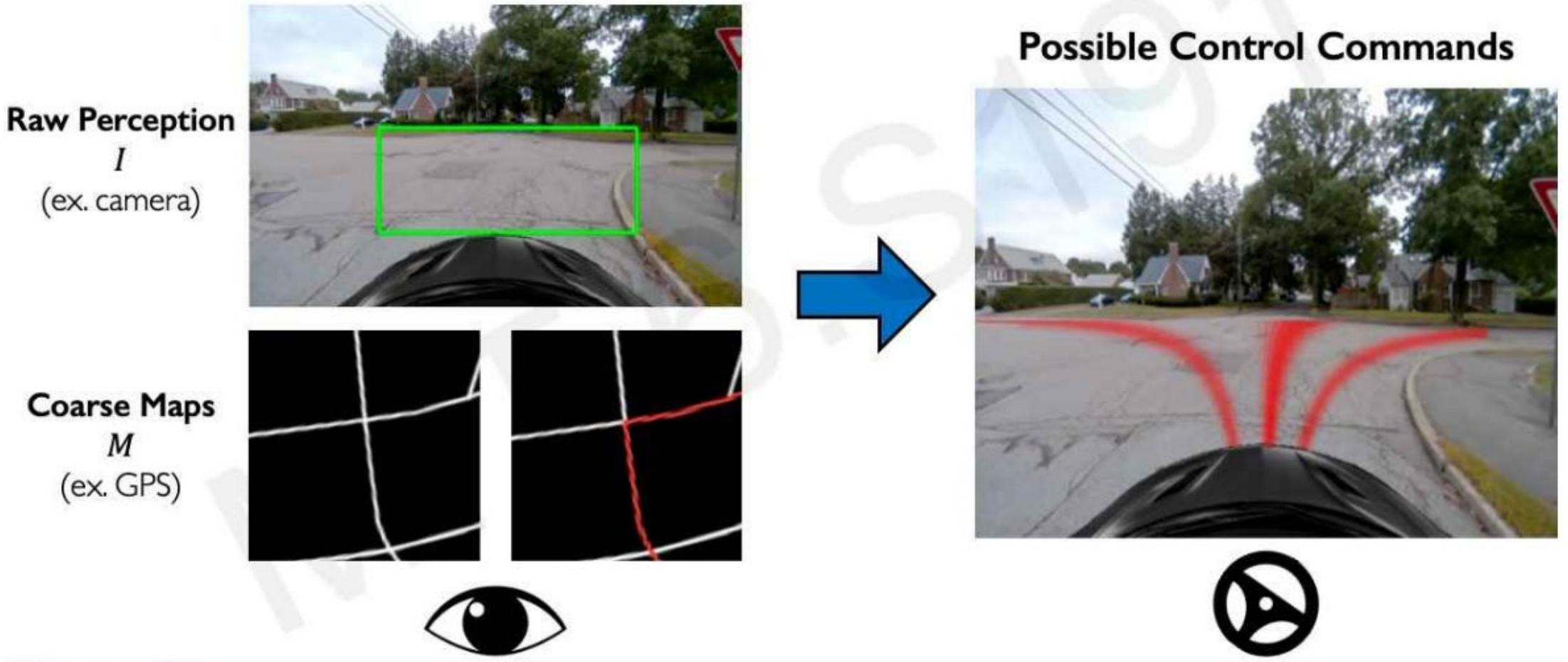
High-res:
D_1 x H/2 x W/2

Predictions:
H x W

© Alexander Amini and Ava Amini

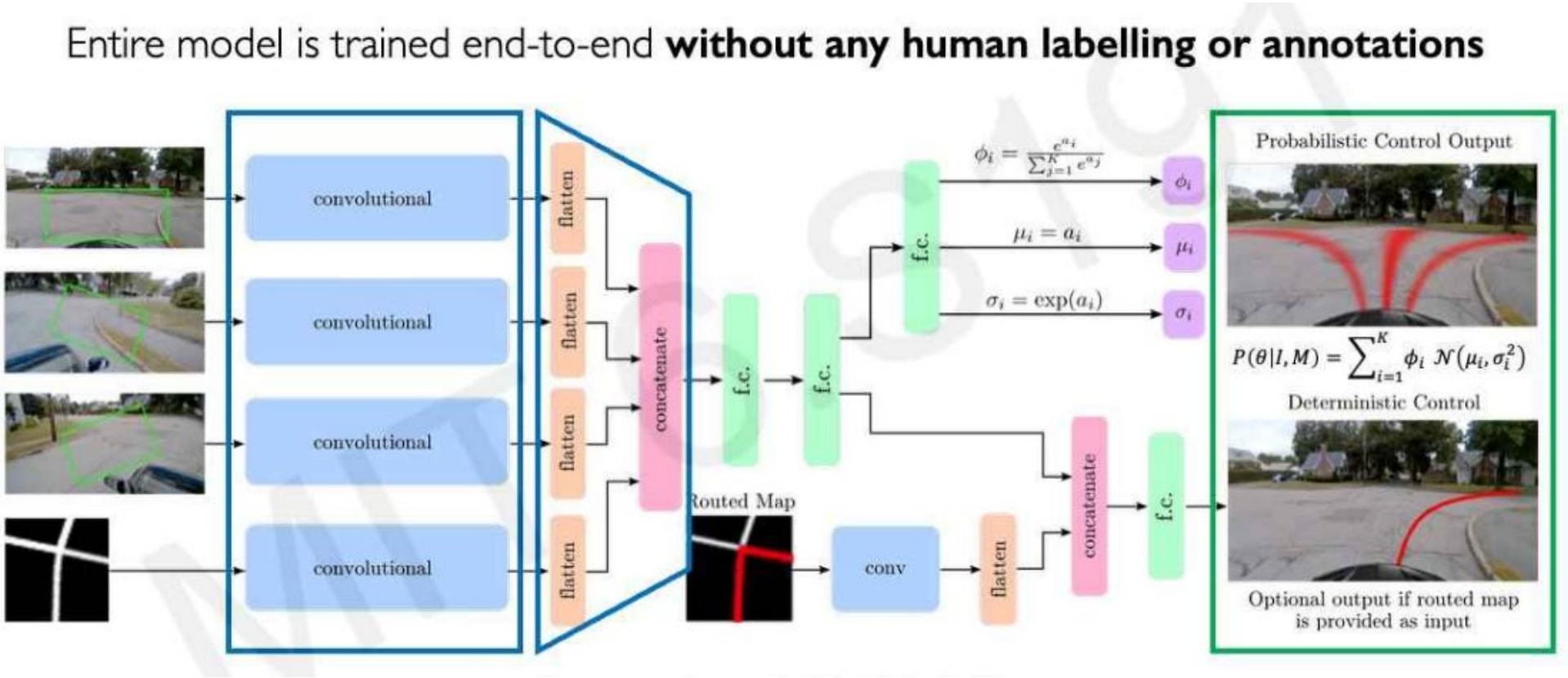
Convolutional Neural Network (CNN)

Continuous Control: Navigation from Vision



© Alexander Amini and Ava Amini

Convolutional Neural Network (CNN)



© Alexander Amini and Ava Amini

Overfitting in CNN

- **Overfitting in CNNs** happens when the model has too many parameters relative to the training data and memorizes examples instead of learning general patterns
- This leads to **poor performance on new/unseen data** despite high training accuracy
- **Regularization** helps prevent overfitting by adding a penalty term to the loss function to discourage overly complex models.
- Common techniques include **L1/L2 regularization** (penalize large weights) and **dropout** (randomly ignore neurons during training to improve robustness).


Q&A